Axion Maintenance System (Outline)

Axion Maintenance System (Outline)

1) Purpose

- Provide **repeatable maintenance operations** for Axion (upgrade, verify, release, recover, etc.).
- Increase confidence using:
  - **Prompts** to declare intent + scope (Operator → Agent interface)
  - **Automation** to enforce deterministic steps + outputs (Machine interface)

2) What it Operates

2.1 Target

- The **Axion System repo** (whatever folder/repo contains `/axion`, UI, tests, etc.)

2.2 Maintenance Modes (LOCKED)

Use the existing locked IDs MM-01 → MM-21 exactly as your canonical mode registry.

Axion Updates Planning

3) Core Concept

3.1 A “Mode Run” is a first-class unit

Every maintenance action is executed as a **Mode Run** with:

- a **Run Spec** (what to do)
- a **Runner** (how it’s done)
- **Outputs** (logs + reports + diffs)
- **Proof** (pass/fail criteria)

3.2 Mix of Prompt + Automation

- **Prompt layer (A):** standardized prompts you drop into the agent to start a run.
- **Automation layer (B):** scripts/commands that actually execute the mode steps consistently.

The agent is the **operator**, not the process source-of-truth.

4) External Control Plane (outside Axion)

Yes — make it.

4.1 Why (in your words)

- You want maintenance to be **managed outside** the system so maintenance can still function even when Axion internals are mid-change/broken.

4.2 What it contains (minimal)

A separate folder or separate repo (either works) that stores:

- Mode registry (MM list)
- Mode runbooks (what steps each mode runs)
- Mode runners (scripts)
- Run output storage (logs/reports)
- A “run router” that maps `MM-xx` → `runner`

4.3 Hard boundary rule

- The external control plane **may call Axion CLI/scripts**, but Axion’s internal control plane must not be required for maintenance execution.

5) File/Folder Layout (recommended structure)

(Place this **outside** `/axion`.)

- `/maintenance/`
  - `/registry/`
    - `modes.json` (MM-01..MM-21, locked IDs)
  - `/runbooks/`
    - `MM-01.health-check.md`
    - `MM-02.full-verify.md`
    - …
  - `/runners/`
    - `mm-01-health-check.(sh|ts)`
    - `mm-02-full-verify.(sh|ts)`
    - …
  - `/runs/` (generated)
    - `YYYYMMDD_HHMMSS_MM-02_<runid>/`
      - `runspec.json`

            - `runlog.md`
            - `report.json`
            - `diff.patch` (optional)
    - `/prompts/`
        - `MAINT_RUN.prompt.md`
        - `MAINT_TRIAGE.prompt.md`
        - `MAINT_UPGRADE.prompt.md`

6) Standard Artifacts per Run (always produced)

6.1 Required

- `RUNSPEC` (json or md): intent, mode_id, scope, constraints, expected proof
- `RUNLOG` (md): commands executed + outputs (timestamps)
- `REPORT` (json): structured results (pass/fail + pointers + metrics)

6.2 Optional (mode-dependent)

- `DIFF` (patch or summary)
- `CHANGELOG` entry (for upgrade/release modes)
- `SNAPSHOT` bundle (for archive/rollback/recovery modes)

7) Execution Contract (how a mode run actually happens)

1. Operator/you selects a mode `MM-xx` from the locked list.
   Axion Updates Planning
2. Agent creates a `RUNSPEC` using the universal prompt.
3. Agent executes **one runner** mapped to that mode.
4. Runner writes outputs to `/maintenance/runs/<run_id>/…`
5. Agent reports the outcome using `report.json` as the source of truth.

8) The Prompt Set (the part you “drop to the agent”)

You’ll maintain **a small set of trigger prompts**. The key one is universal:

8.1 Universal Prompt: `MAINT_RUN`

Inputs it must force:

- `mode_id` (MM-01..MM-21)

- `scope` (paths, or “all”)
- `constraints` (stop-on-fail, locked folders, no refactors, etc.)
- `expected_proof` (which checks must pass)
- `output_dir` (run folder path)

8.2 Upgrade Prompt: `MAINT_UPGRADE`

Adds:

- change summary
- dependency closure block (deps + revdeps + transitive)
- migration required? (y/n)
- version bump plan

8.3 Triage Prompt: `MAINT_TRIAGE`

Adds:

- symptom
- last known good run id
- reproduction steps
- minimize failing case

9) Mode Implementation Strategy

- Start by implementing runners for the most-used modes:
  - MM-01, MM-02, MM-05, MM-06, MM-12, MM-14, MM-20, MM-21
- Every runner must be:
  - non-interactive (CI-safe)
  - deterministic
  - strict about writing `REPORT.json`

Purpose (Revised Draft)

Purpose (Revised Draft)

1.1 Mission

Create an **external maintenance layer** that lets you operate Axion safely over time (verify, upgrade, release, recover) with **high confidence** and **low drift**, even while Axion internals are being replaced.

1.2 What “high confidence” means (definition)

A maintenance action is considered valid only if it produces:

- a **Run Spec** (what was intended)
- a **Run Log** (what actually ran)
- a **Machine Report** (pass/fail + pointers + metrics)
- and (when applicable) a **Diff / Changelog / Snapshot**

No “done” state exists without those outputs.

1.3 Non-goals (explicit)

- Not a replacement for Axion’s internal control plane.
- Not a “smart” agent that invents steps.
- Not a UI-first system (UI can be added later, but execution is the priority).

1.4 Why external (boundary intent)

The maintenance system must remain usable when:

- Axion templates/config/registry are mid-change
- Axion verification is failing
- the UI is being swapped

- the repo is in a partially broken state

So the maintenance control plane:

- **can call Axion**, but **must not require Axion's internals** to function.

1.5 Core outcomes (what you get)

- Repeatable **MM-01..MM-21** runs with proof.
- Deterministic "operator experience" via copy/paste prompts **and** runners.
- A persistent **audit trail** of every maintenance action.

Status: Section 1 is ready to lock.

What It Operates (Revised Draft)

What It Operates (Revised Draft)

2.1 Targets

The Axion Maintenance System operates on **one or more target workspaces** (usually a repo), each treated as a “maintenance subject.”

A target can be:

- **Axion Core Repo** (engine/templates/registry/config/tests/UI)
- **Axion UI Repo** (if separated)
- **Axion Template Library Repo** (if separated)
- Any repo/workspace that must stay compatible with Axion contracts

2.2 Target Identification (how a repo is recognized)

Each target workspace must expose a minimal “target identity” so runners don’t guess:

- `target_id` (string)
- `target_root` (path)
- `target_type` (axion_core | ui | templates | other)
- `runtime_contract` (how to run/verify in that repo: scripts/entrypoints)
- optional: `expected_layout` (paths that should exist)

2.3 Scope Model (what a mode run is allowed to touch)

Every mode run must declare one of:

- **Scope = Whole Target** (entire repo)
- **Scope = Path Set** (explicit paths)

- **Scope = Component Set** (ENGINE / TEMPLATE / REGISTRY / CONFIG / UI / TESTS etc.)
- **Scope = Diff Range** (e.g., “since last snapshot/run id”)

No mode run is allowed to proceed without an explicit scope.

2.4 Canonical Maintenance Modes (Locked)

Operations are executed only through the locked mode registry:

- **MM-01 … MM-21** (the list we locked earlier)

A mode run must specify exactly one `mode_id` from this registry.

2.5 Mode-to-Target Compatibility

Not every mode applies to every target. Each mode has:

- `allowed_target_types` (e.g., MM-14 Release Packaging may be core-only)
- `required_capabilities` (e.g., “node”, “git”, “axion-cli”, “tests”)

If a target lacks required capabilities, the run fails fast with a clear report.

2.6 Inputs the Maintenance System may consume

- repo filesystem (source code, templates, config)
- git metadata (optional but strongly supported: branch, commit, diff)
- environment variables (read-only unless explicitly allowed per mode)
- previous run artifacts (runspec/report/snapshot)

2.7 Outputs location (high level)

All outputs are written **outside** the Axion internal folders, under the maintenance control plane run directory (we’ll detail paths in Section 5/6):

- runspec
- runlog
- report
- diffs/snapshots when applicable

Status: Section 2 is ready to lock.

Mode Run is the unit of work

Mode Run is the unit of work

A **Mode Run** is the only valid way maintenance actions happen.

A mode run is defined by:

- **Intent**: `mode_id` (MM-xx)
- **Target**: which workspace/repo
- **Scope**: what parts are in-bounds
- **Constraints**: what is forbidden/required
- **Steps**: the deterministic runner sequence
- **Proof**: machine-verifiable pass/fail

3.2 Two-layer execution model (Mix of A + B)

You're locking a hybrid model:

Layer A — Prompted Operation (Agent Interface)

The agent receives a **trigger prompt** and produces (or updates) a `RUNSPEC`.

- Prompts are standardized and short.
- Prompts never contain "the whole process."
- Prompts force explicit scope + constraints + expected proof.

Layer B — Deterministic Execution (Runner Interface)

A runner executes the mode in a consistent way:

- same step order
- same required outputs
- same failure semantics

- same report format

The runner is the source of truth for “how the mode runs.”

3.3 Agent role (operator, not author of the process)

The agent is allowed to:

- create/edit `RUNSPEC`
- invoke the runner
- interpret the `REPORT`
- propose a fix plan (and optionally apply fixes if the mode allows edits)

The agent is not allowed to:

- improvise steps outside the runner
- declare success without required proof outputs

3.4 Determinism and auditability rules

A mode run is only “complete” when:

- `REPORT.json` exists
- `REPORT.status = PASS`
- `REPORT.proof` contains all required proof items for that mode
- outputs are stored under a unique run directory

3.5 Stop-on-fail as default behavior

Default policy:

- **stop-on-fail = true**

- failures produce pointers + a partial runlog + report with `FAIL`
- no further steps are executed unless a mode explicitly allows “continue-on-fail”

3.6 Idempotence target

Runners should be designed to be **re-runnable**:

- re-running the same spec should not corrupt state
- outputs go to a new run folder each time (or a “rerun_of” link in spec/report)

Status: Section 3 is ready.

External Control Plane (Revised Draft)

Section 4 — External Control Plane (Revised Draft)

4.1 Definition

The **Axion Maintenance Control Plane** is a separate management layer that:

- owns the **mode registry**
- owns the **runbooks/runners**
- owns **run output storage**
- orchestrates runs against target workspaces

It lives **outside** Axion's internal control plane.

4.2 Boundary rule (hard requirement)

The maintenance control plane:

- **may invoke Axion tooling** (CLI/scripts) when available
- **must not require Axion's internal control plane to operate**

Meaning: even if Axion's templates/registry/config are broken mid-upgrade, you can still run:

- triage
- recovery
- rebuild indexes
- snapshot
- compatibility checks

4.3 Independence model (what "not involved" means)

To keep Axion internal CP uninvolved:

- maintenance state is stored in `/maintenance/...` (or separate repo), not under `/axion/...`
- maintenance runners do not depend on Axion registry/config to *decide what steps to take*
- where Axion CP exists, it is treated as a **black-box capability** (call it if it works; don't rely on it)

4.4 Control plane responsibilities (locked)

1. **Mode Registry Management**
    - stores MM-01..MM-21 definitions

    - mode applicability rules (allowed targets/capabilities)
2. **Runner Execution**
    - maps `mode_id` → runner entrypoint
    - validates runspec before executing
    - enforces output writing and report schema
3. **Run Storage**
    - write-once run folders
    - immutable run artifacts after completion (optional but recommended)
4. **Audit Trail**
    - run index (searchable)
    - link run → target → commit hash (if git present)
5. **Policy Enforcement**
    - stop-on-fail default
    - locked folders per mode
    - override handling (if you allow it)

4.5 Where it lives (deployment options)

- **Option 1: Folder inside the repo root**
  `/maintenance/*` (simplest operationally)
- **Option 2: Separate repo**
  `axion-maintenance/` that targets Axion repos via paths

Both satisfy "outside Axion internal CP." The key is the **boundary**: not inside `/axion/*` and not dependent on internal maps to function.

4.6 Minimal control plane components (MVP)

- Mode registry file
- Runspec schema + validator
- Runner dispatcher ("mode run" command)
- Standard run folder writer
- Report schema writer

Status: Section 4 is ready.

Next section is **Section 5 — File/Folder Layout** (exact structure + naming + IDs).

File/Folder Layout (Revised Draft)

Section 5 — File/Folder Layout (Revised v2, with registry schemas)

5.1 Root placement (locked)

`/maintenance/` at repo root.

5.2 Canonical directory structure (locked)

(unchanged)

5.3 Run ID format (locked)

(unchanged)

5.4 Registry files (expanded + locked schemas)

5.4.1 `registry/modes.json` (LOCKED structure)

Purpose: map `MM-xx` → runner + applicability + required proof.

Minimum fields per mode:


```
{
 "version": "1.0",
 "modes": [
   {
     "mode_id": "MM-02",
     "name": "Full Verification Run",
     "description": "Runs the full gate/test verification pipeline and emits a report.",
     "runner": {
       "entrypoint": "runners/mm-02-full-verify.js",
       "kind": "node|shell|python"
     },
```

```
    "allowed_target_types": ["axion_core", "ui", "templates", "other"],
    "required_capabilities": ["node", "npm"],
    "default_constraints": {
      "stop_on_fail": true,
      "allow_writes": false,
      "allow_network": false,
      "locked_paths": []
    },
    "required_proof": [
      { "kind": "command_pass", "value": "npm test" }
    ],
    "outputs": {
      "diff": false,
      "snapshot": false,
      "changelog": false
    }
  }
]
}
```

Rules:

- `mode_id` must be one of MM-01..MM-21 (locked list).
- `runner.entrypoint` is relative to `/maintenance/`.
- `default_constraints` are applied unless overridden by runspec (if policy allows).

5.4.2 `registry/targets.json` (LOCKED structure)

Purpose: define known targets so runs don't guess.

```
{
 "version": "1.0",
 "targets": [
   {
     "target_id": "axion-core",
     "root": ".",
     "type": "axion_core",
     "runtime_contract": {
       "start": "npm run dev",
       "verify": "npm test",
       "build": "npm run build"
     },
     "capabilities_hint": ["node", "npm", "git"],
     "expected_paths": ["axion", "client", "server"]
   }
 ]
}
```

Rules:

- `root` is path relative to repo root (or absolute allowed later).
- `runtime_contract` strings are commands; runners choose which to execute.

5.4.3 `registry/policies.json` (LOCKED structure)

Purpose: global enforcement rules.

```
{
 "version": "1.0",
 "defaults": {
   "stop_on_fail": true,
   "allow_network": false
 },
 "overrides": {
   "allow_runspec_override_constraints": true,
   "allow_runspec_override_required_proof": false
 },
 "locked_paths_global": [
   ".git/",
   "maintenance/runs/"
 ],
 "write_rules": {
   "when_allow_writes_false": {
     "allowed_write_roots": ["maintenance/runs/"]
   }
 }
}
```

Rules:

- `maintenance/runs/` is always write-only.
- You can later add per-mode locked paths, but global locks always apply.

5.5 Generated outputs are write-only (locked)

(unchanged)

5.6 Language choice for runners (not locked yet)

(unchanged)

✅ Section 5 now contains the concrete registry/policy definitions without adding Section 10.

Section 7 — Execution Contract (Revised v2, with resolution + enforcement rules)

7.2 Dispatcher responsibilities (expanded)

Add these two required steps:

2a. Resolve mode definition

- Load `/maintenance/registry/modes.json`
- Find the `mode_id`
- If not found → FAIL with `report.status=FAIL`

2b. Merge constraints + proofs

- Start with:
  - `policies.json defaults`
    - `modes.json default_constraints`
- Apply `runspec.constraints` only if `policies.overrides.allow_runspec_override_constraints=true`
- Set `expected_proof` from:

    - `modes.json required_proof` (always)
    - `runspec.expected_proof` can only *add* items if override is allowed (v1 default: do not allow replacing required proof)

7.3 Capability gating (new locked rule)

Before running the runner:

- Dispatcher checks `required_capabilities` from the mode definition.
- If missing → FAIL early with clear report message.

7.7 Write enforcement (expanded)

When `allow_writes=false`:

- Dispatcher must enforce:
    - only writes under `maintenance/runs/<run_id>/`
    - and any other allowed write roots from `policies.json`
- Post-run: dispatcher checks for changes outside allowed write roots (git diff or filesystem snapshot strategy) and fails if violated.

Standard Artifacts per Run

Section 6 — Standard Artifacts per Run (Revised Draft)

Every mode run produces a **standard artifact set**. This is what makes “confidence” measurable.

6.1 Required artifacts (locked)

These must exist for **every** MM-xx run:

1. `runspec.json` — *intent + boundaries*
2. `runlog.md` — *what executed (human readable)*
3. `report.json` — *results + proof (machine readable)*

A run is invalid if any of the three is missing.

6.2 `runspec.json` (minimum required fields)

```
{
 "run_id": "20260226_083015_local_MM-02_a1b2c3d4",
 "mode_id": "MM-02",
 "target": {
   "target_id": "axion-core",
   "root": ".",
   "type": "axion_core"
 },
 "scope": {
   "type": "all|paths|components|diff_range",
   "paths": [],
   "components": [],
   "diff": { "from": "", "to": "" }
 },
 "constraints": {
   "stop_on_fail": true,
   "locked_paths": [],
   "allow_writes": true,
   "allow_network": false
 },
 "requested_outputs": {
   "diff": false,
   "snapshot": false,
```

```
  "changelog": false
 },
 "expected_proof": [
  { "kind": "command_pass", "value": "axion verify" }
 ],
 "metadata": {
  "env": "local|ci|prod",
  "requested_by": "user|agent",
  "notes": ""
 }
}
```

Notes:

- `scope.type` forces explicit boundaries.
- `constraints` is how you keep the agent from going rogue.
- `expected_proof` is how you prevent “it seems fine” conclusions.

6.3 `runlog.md` (required sections)

Must include these headers, in this order:

1. Run Summary (run_id, mode_id, target, scope)
2. Preconditions (capabilities detected, versions)
3. Steps Executed (timestamped)
4. Commands + Output (trimmed, with pointers to full logs if needed)
5. Artifacts Written (paths)
6. Result (PASS/FAIL + one-line reason)

6.4 `report.json` (minimum required fields)

```
{
 "run_id": "20260226_083015_local_MM-02_a1b2c3d4",
 "mode_id": "MM-02",
 "status": "PASS|FAIL|WARN",
 "target": { "target_id": "axion-core", "commit": "", "branch": "" },
```

```
 "started_at": "2026-02-26T08:30:15-05:00",
 "finished_at": "2026-02-26T08:33:02-05:00",
 "results": [
   {
     "check_id": "verify",
     "status": "PASS|FAIL|WARN",
     "summary": "",
     "pointers": [
       { "path": "", "line": 0, "message": "" }
     ]
   }
 ],
 "proof": [
   { "kind": "command_pass", "value": "axion verify", "evidence_path": "runlog.md" }
 ],
 "artifacts": {
   "runspec": "runspec.json",
   "runlog": "runlog.md",
   "diff": "",
   "snapshot": "",
   "changelog": ""
 }
}
```


6.5 Optional artifacts (mode-dependent)

Stored under `/maintenance/runs/<run_id>/artifacts/`

- `diff.patch` or `diff.md`
- `snapshot.zip`
- `changelog.md`
- `repair_plan.md` (common for triage/recovery)

6.6 Schema enforcement (locked)

- `runspec.json` and `report.json` must validate against:
    - `/maintenance/schemas/runspec.schema.json`

- `/maintenance/schemas/report.schema.json`

Runners must fail if they can’t write valid JSON matching schema.

Status: Section 6 is read

Execution Contract (Revised Draft)

Section 7 — Execution Contract (Revised Draft)

This section defines *exactly* how modes get executed in a way that is consistent and agent-safe.

7.1 One entrypoint (locked)

All runs are started through one dispatcher command:

- `maintenance run <mode_id> --spec <path_to_runspec>`

Implementation can be Node or shell, but the interface stays the same.

7.2 Dispatcher responsibilities (locked)

The dispatcher must:

1. **Load + validate** `runspec.json` against schema
2. **Resolve the runner** for `mode_id` using `registry/modes.json`
3. **Create the run directory** `/maintenance/runs/<run_id>/`
4. **Copy/finalize** the spec into the run folder as `runspec.json` (source of truth for that run)
5. **Execute the runner** with:
    - `MODE_ID`
    - `RUN_ID`
    - `TARGET_ROOT`
    - `RUN_DIR`
    - `SCOPE`
    - `CONSTRAINTS`

6. **Enforce constraints** at the dispatcher level when possible:
    - stop-on-fail default
    - locked path checks (pre + post)
7. **Guarantee outputs**:
    - ensure `runlog.md` + `report.json` exist even on failure
8. **Return correct exit code**:
    - PASS → exit 0
    - FAIL → exit 1
    - WARN → exit 2 (optional; still non-zero)

7.3 Runner responsibilities (locked)

A mode runner must:

- perform capability checks (node, git, axion cli, etc.)
- execute the mode's steps (from runbook)
- stream logs into `runlog.md`
- write `report.json` with:
    - status
    - check results
    - pointers on failure
    - proof items

A runner must never claim PASS without writing proof items.

7.4 Runner lifecycle (locked)

1. **Preflight**
    - confirm target exists
    - confirm required capabilities
    - confirm scope is valid
2. **Execute steps**
3. **Collect outputs**
4. **Write report**
5. **Postflight**
    - verify locked paths not modified (if configured)
    - verify requested artifacts exist

7.5 Failure semantics (locked)

- If any required check fails:
    - runner sets `report.status = FAIL`
    - runner writes pointers
    - dispatcher stops further steps unless mode explicitly allows continue
- Partial output is allowed, but the standard artifacts must exist.

7.6 Agent trigger pattern (how prompts connect to execution)

The agent flow is standardized:

1. Agent receives a trigger prompt (e.g., MAINT_RUN)
2. Agent generates `runspec.json`

3. Agent calls dispatcher:
   `maintenance run MM-xx --spec /path/to/runspec.json`

4. Agent reads `report.json` as truth

5. Agent returns a human summary + next actions (if fail)

7.7 Write permissions model (locked)

Modes declare `constraints.allow_writes`.

- `allow_writes = false` means:
  - only allowed writes are under `/maintenance/runs/<run_id>/…`
  - repo files must not change
- `allow_writes = true` means:
  - runner (and agent, if allowed by mode) may apply changes in-scope
  - changes must be captured in `diff` artifacts when requested

7.8 “No internal Axion CP dependency” enforcement

Even if Axion’s internal CP is broken:

- dispatcher still runs
- run artifacts still write
- triage/recovery modes can still execute using filesystem + git
- any call to Axion commands is treated as optional capability that can fail gracefully (mode-dependent)

Prompt Set (Revised Draft)

Section 8 — Prompt Set (Revised v2, aligned to `modes.json/targets.json/policies.json`)

This revision makes the prompts **guarantee** they can be executed without the agent inventing structure.

8.1 Prompt philosophy (locked)

- Prompts must produce a **valid `runspec.json`** that the dispatcher can validate.
- Prompts must reference **existing IDs**:
  - `mode_id` must exist in `maintenance/registry/modes.json`
  - `target_id` must exist in `maintenance/registry/targets.json` (unless you allow ad-hoc targets later)
- Prompts do not redefine procedures. They only set **scope + boundaries + run options**.

8.2 Canonical prompts (locked list)

(unchanged)

1. `MAINT_RUN`
2. `MAINT_UPGRADE`
3. `MAINT_TRIAGE`
4. `MAINT_RECOVERY`

8.3 Output contract for all prompts (locked)

Every prompt must force the agent to output, in order:

1. **`runspec.json`** (schema-valid)

2. **Dispatcher command**: `maintenance run <mode_id> --spec <path>`

3. **Post-run read step**: open `maintenance/runs/<run_id>/report.json` and summarize results from it (no “vibes”).

8.4 `MAINT_RUN` (template, updated)

Inputs:

- `mode_id` (MM-01..MM-21)
- `target_id` (from targets.json)
- `scope.type` = all|paths|components|diff_range
- Optional overrides allowed by policies:
    - constraints (only if policies allow)
    - requested_outputs (diff/snapshot/changelog)

Hard rule inside the prompt:
If policies disallow overriding required proof, the agent must **copy required proof from modes.json** into `runspec.expected_proof` without modification.

8.5 `MAINT_UPGRADE` (template, updated)

This prompt is for writing *planning artifacts* plus a run.

Required additional artifact (written into the run folder):

- `artifacts/upgrade_plan.md`

And the prompt must ensure:

- `mode_id` is one of:
    - MM-05 Planned Upgrade

    - MM-06 Hotfix
    - MM-08 Breaking Upgrade

And it must include:

- dependency closure block
- compatibility classification
- migration yes/no
- version bump plan

8.6 `MAINT_TRIAGE` (template, updated)

Produces:

- runspec (MM-20)
- `artifacts/triage.md` with:
    - repro steps
    - suspected scope
    - minimal failing case
    - last known good commit/run_id

8.7 `MAINT_RECOVERY` (template, updated)

Produces:

- runspec (MM-21)
- `artifacts/recovery_plan.md` with:
    - restore goal

    - destructive actions allowed
    - optional snapshot/run_id inputs

8.8 Safety constraint baked into prompts (locked)

Each prompt must instruct the agent:

- Do not edit files outside `scope` and outside allowed write rules.
- If `allow_writes=false`, do not propose or apply changes to repo files.

Status: Section 8 updated and consistent with registries/policies.

Mode Implementation Strategy

Section 9 — Mode Implementation Strategy (Revised v2, registry-driven)

This revision ties implementation directly to `modes.json`, `targets.json`, and `policies.json` so nothing is "implied."

9.1 Phases (locked)

(kept the same phase breakdown) but with one enforcement rule:

Every mode is considered "implemented" only when:

- it exists in `registry/modes.json`
- its runner entrypoint exists in `/maintenance/runners/`
- it has a runbook in `/maintenance/runbooks/`
- it produces schema-valid `report.json` with required proof

9.2 Phase 0 — Foundation (expanded + locked deliverables)

Deliverables:

1. `/maintenance/registry/{modes.json,targets.json,policies.json}`
2. `/maintenance/schemas/{runspec.schema.json,report.schema.json}`
3. `/maintenance/runners/dispatcher.(ts|js|sh)` implementing Section 7
4. A "hello-world" runner (MM-01 minimal) that writes all standard artifacts
5. A `runs/` folder generator with run_id formatting rules

9.3 Phase 1 — Core modes (expanded with capability + proof defaults)

For each mode in Phase 1, we define in `modes.json`:

- `required_capabilities`
- `default_constraints`
- `required_proof`
- `outputs` toggles

And then implement runner + runbook.

Phase 1 modes (unchanged list):

- MM-01, MM-02, MM-12, MM-18, MM-20, MM-21

9.4 Runner standards (locked, but more explicit)

Each runner must implement these internal steps:

1. **Preflight**
    - load target runtime contract from `targets.json`
    - verify required capabilities (or accept dispatcher's result)
2. **Execute**
    - run commands described by the runbook (or embedded steps)
3. **Collect evidence**
    - capture command exit codes + relevant output lines
4. **Write outputs**
    - runlog.md and report.json always
5. **Proof mapping**
    - convert evidence → `report.proof[]` items matching `modes.json` `required_proof`

9.5 Enforcement rollout (locked)

Enforcement is staged but deterministic:

- **Stage A (MVP):** validate runspec + write report + stop-on-fail
- **Stage B:** locked-path checks (pre/post) using git diff when possible
- **Stage C:** scope enforcement (paths/components) at dispatcher level
- **Stage D:** full “allow_writes=false” filesystem enforcement for non-git targets (optional later)

9.6 “No internal Axion CP dependency” handling (locked)

For every mode runner:

- Steps are defined by maintenance runbooks + runtime contract.
- Axion internal tooling is optional capability. If it fails:
    - runner records a FAIL result with pointers
    - recovery/triage modes still function (they are not dependent on those tools)

Status: Section 9 v2 is complete.

Implementation

Axion Maintenance System — Phase Plan

Phase 0 — Foundation (Make Modes Runnable)

Objective: Build the external control plane so *any* mode can be executed deterministically and always produces proof artifacts.

P0.1 Control Plane Skeleton

- Create folder skeleton:
  - `/maintenance/registry/`
  - `/maintenance/runbooks/`
  - `/maintenance/runners/`
  - `/maintenance/prompts/`
  - `/maintenance/schemas/`
  - `/maintenance/runs/` (generated)

P0.2 Contracts (Schemas)

- Implement:
  - `schemas/runspec.schema.json`
  - `schemas/report.schema.json`

P0.3 Registries (Source-of-Truth Lists)

- Implement:
  - `registry/policies.json`
  - `registry/targets.json`
  - `registry/modes.json` (MM-01..MM-21; stubs allowed initially)

P0.4 Dispatcher (Deterministic Execution Entry)

- Implement `runners/dispatcher.(js|ts|sh)`:
  - validate runspec
  - resolve mode + target
  - merge policies/constraints
  - capability checks
  - create run folder
  - execute runner
  - guarantee run artifacts

    - enforce stop-on-fail exit codes

P0.5 Runner Baseline (Stubs + 1 Real Mode)

- Implement:
    - `runners/mm-stub-not-implemented.*` (clean FAIL with artifacts)
    - `runners/mm-01-health-check.*` (first PASS-capable mode)

P0.6 Enforcement (MVP)

- Implement enforcement utilities:
    - `allow_writes=false` enforcement (MVP: `git status --porcelain` / diff check)
    - global locked paths enforcement (minimal first pass)

P0.7 Prompt Pack (Agent Triggers)

- Create prompt files:
    - `prompts/MAINT_RUN.prompt.md`
    - `prompts/MAINT_UPGRADE.prompt.md`
    - `prompts/MAINT_TRIAGE.prompt.md`
    - `prompts/MAINT_RECOVERY.prompt.md`
- Locked behavior: each prompt outputs
    - `runspec.json`
    - dispatcher command
    - instruction to read `report.json` and summarize strictly from it

Phase 0 Done When:

- You can run `MM-01` and get `runspec/runlog/report` with PASS/FAIL deterministically.
- `allow_writes=false` is enforced.

Phase 1 — Core Operations (Run, Verify, Diagnose, Recover)

Objective: Implement the maintenance spine: verify system health, compatibility, rebuild, triage, recovery.

P1.1 Implement core runners + runbooks

- MM-02 — Full Verification Run
- MM-12 — Rebuild Indexes/Registries
- MM-18 — Workspace Compatibility Check
- MM-20 — Failure Triage Mode
- MM-21 — Emergency Recovery Mode

Phase 1 Done When:

- Each mode has:
  - `modes.json` entry
  - runner file
  - runbook file
  - PASS/FAIL report with required proof

Phase 2 — Change & Release (Safe Upgrades + Packaging)

Objective: Controlled updates and releases with dependency closure + proofs.

P2.1 Implement change modes

- MM-05 — Planned Upgrade
- MM-06 — Hotfix

P2.2 Implement packaging/rollback modes

- MM-16 — Snapshot / Archive
- MM-14 — Release Packaging
- MM-15 — Rollback / Revert Release

Phase 2 Done When:

- You can upgrade safely, produce diffs/changelogs, package a release, and rollback with proof.

Phase 3 — Full Coverage (Remaining Maintenance Modes)

Objective: Complete the mode set and tighten governance/compliance behavior.

P3.1 Implement remaining modes (recommended order)

- MM-03 — Coverage Audit
- MM-04 — Drift Detection
- MM-17 — Dependency Refresh
- MM-19 — CI Sanity Run
- MM-13 — Registry Integrity Cleanup
- MM-11 — Data Repair / Salvage
- MM-10 — Backcompat Validation
- MM-09 — Artifact Migration
- MM-07 — Deprecation Rollout
- MM-08 — Breaking Upgrade (Major Version)

Phase 3 Done When:

- MM-01..MM-21 all have real runners (no stubs).
- Proof and constraints are enforced consistently.

Execution Model Label (for your docs)

- **Prompt Layer (A):** Prompt Pack (Phase 0.7) produces runspec + dispatch command.
- **Automation Layer (B):** Dispatcher + runners (Phase 0.4 onward) execute deterministically and output proof artifacts.

If you want, I can format this into your Axion doc style (IDs, headers, “LOCKED” blocks) but this is already clean for documentation labeling.

-

Foundation

Foundation (required before any mode works)

1. **Create folder skeleton**

- `/maintenance/registry/`
- `/maintenance/runbooks/`
- `/maintenance/runners/`
- `/maintenance/prompts/`
- `/maintenance/schemas/`
- `/maintenance/runs/` *(generated; keep empty or gitignore)*

2. **Implement schemas**

- `schemas/runspec.schema.json`
- `schemas/report.schema.json`

3. **Implement registries**

- `registry/modes.json` (MM-01..MM-21 entries, even if some are “stubbed” initially)
- `registry/targets.json` (at least 1 target: axion-core)
- `registry/policies.json` (defaults + override rules + global locks)

4. **Implement dispatcher**

- `runners/dispatcher.(ts|js|sh)` supporting:
    - validate runspec schema
    - resolve mode from modes.json
    - resolve target from targets.json

    - merge constraints/policies
    - capability checks
    - create run folder
    - execute runner entrypoint
    - guarantee standard artifacts exist
    - enforce stop-on-fail exit codes

5. **Implement run writer utilities**

- run_id generator (format locked)
- write `runspec.json` into run folder
- append to `runlog.md` with timestamps
- write schema-valid `report.json`

6. **Implement enforcement utilities**

- locked path check (global + per-mode)
- allow_writes enforcement:
    - MVP: `git diff` check after run
    - fallback: filesystem snapshot/mtime scan (optional later)

7. **Add gitignore rules**

- ignore `/maintenance/runs/**` (optional if you want runs persisted locally only)
- ignore temp artifacts if needed

Create folder skeleton

Create folder skeleton

Create these folders at repo root:

- `maintenance/registry/`
- `maintenance/runbooks/`
- `maintenance/runners/`
- `maintenance/prompts/`
- `maintenance/schemas/`
- `maintenance/runs/`

One-shot command (safe)

mkdir -p maintenance/{registry,runbooks,runners,prompts,schemas,runs}

Acceptance criteria (must be true)

- All six directories exist exactly at those paths.

Piece 0.2 — Add gitignore rule for generated runs

Add this line to your `.gitignore`:

maintenance/runs/**

Acceptance criteria

- `maintenance/runs/` stays empty in git (runs are generated only).

Implement Schemas

Implement Schemas (runspec + report)

We'll do **runspec.schema.json** first, then **report.schema.json**.

0.3.1 Create `maintenance/schemas/runspec.schema.json` (LOCKED v1)

Save this exact JSON:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "axion-maintenance:runspec.schema.json",
 "title": "Axion Maintenance Run Spec",
 "type": "object",
 "additionalProperties": false,
 "required": [
   "run_id",
   "mode_id",
   "target",
   "scope",
   "constraints",
   "requested_outputs",
   "expected_proof",
   "metadata"
 ],
 "properties": {
   "run_id": {
     "type": "string",
     "minLength": 10
   },
   "mode_id": {
     "type": "string",
     "pattern": "^MM-(0[1-9]|1[0-9]|2[0-1])$"
   },
   "target": {
     "type": "object",
     "additionalProperties": false,
     "required": ["target_id", "root", "type"],
     "properties": {
       "target_id": { "type": "string", "minLength": 1 },
       "root": { "type": "string", "minLength": 1 },
       "type": {
```

```
      "type": "string",
      "enum": ["axion_core", "ui", "templates", "other"]
    }
  }
},
"scope": {
  "type": "object",
  "additionalProperties": false,
  "required": ["type"],
  "properties": {
    "type": {
      "type": "string",
      "enum": ["all", "paths", "components", "diff_range"]
    },
    "paths": {
      "type": "array",
      "items": { "type": "string", "minLength": 1 },
      "default": []
    },
    "components": {
      "type": "array",
      "items": { "type": "string", "minLength": 1 },
      "default": []
    },
    "diff": {
      "type": "object",
      "additionalProperties": false,
      "required": ["from", "to"],
      "properties": {
        "from": { "type": "string" },
        "to": { "type": "string" }
      },
      "default": { "from": "", "to": "" }
    }
  }
},
"constraints": {
  "type": "object",
  "additionalProperties": false,
  "required": ["stop_on_fail", "locked_paths", "allow_writes", "allow_network"],
  "properties": {
    "stop_on_fail": { "type": "boolean" },
    "locked_paths": {
      "type": "array",
```

```
        "items": { "type": "string", "minLength": 1 }
      },
      "allow_writes": { "type": "boolean" },
      "allow_network": { "type": "boolean" }
    }
  },
  "requested_outputs": {
    "type": "object",
    "additionalProperties": false,
    "required": ["diff", "snapshot", "changelog"],
    "properties": {
      "diff": { "type": "boolean" },
      "snapshot": { "type": "boolean" },
      "changelog": { "type": "boolean" }
    }
  },
  "expected_proof": {
    "type": "array",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "required": ["kind", "value"],
      "properties": {
        "kind": { "type": "string", "minLength": 1 },
        "value": { "type": "string", "minLength": 1 }
      }
    },
    "minItems": 1
  },
  "metadata": {
    "type": "object",
    "additionalProperties": false,
    "required": ["env", "requested_by", "notes"],
    "properties": {
      "env": { "type": "string", "enum": ["local", "ci", "prod"] },
      "requested_by": { "type": "string", "enum": ["user", "agent"] },
      "notes": { "type": "string" }
    }
  },
  "rerun_of": {
    "type": "string",
    "minLength": 1
  }
}
```

}

Acceptance criteria

- File exists at `maintenance/schemas/runspec.schema.json`
- Valid JSON
- `mode_id` pattern only allows MM-01..MM-21

Create `maintenance/schemas/report.schema.json` (LOCKED v1)

Save this exact JSON:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "axion-maintenance:report.schema.json",
 "title": "Axion Maintenance Report",
 "type": "object",
 "additionalProperties": false,
 "required": [
   "run_id",
   "mode_id",
   "status",
   "target",
   "started_at",
   "finished_at",
   "results",
   "proof",
   "artifacts"
 ],
 "properties": {
   "run_id": { "type": "string", "minLength": 10 },
   "mode_id": {
     "type": "string",
     "pattern": "^MM-(0[1-9]|1[0-9]|2[0-1])$"
   },
   "status": { "type": "string", "enum": ["PASS", "FAIL", "WARN"] },
   "target": {
     "type": "object",
     "additionalProperties": false,
```

```
  "required": ["target_id", "commit", "branch"],
  "properties": {
    "target_id": { "type": "string", "minLength": 1 },
    "commit": { "type": "string" },
    "branch": { "type": "string" }
  }
},
"started_at": { "type": "string", "minLength": 1 },
"finished_at": { "type": "string", "minLength": 1 },
"results": {
  "type": "array",
  "minItems": 1,
  "items": {
    "type": "object",
    "additionalProperties": false,
    "required": ["check_id", "status", "summary", "pointers"],
    "properties": {
      "check_id": { "type": "string", "minLength": 1 },
      "status": { "type": "string", "enum": ["PASS", "FAIL", "WARN"] },
      "summary": { "type": "string" },
      "pointers": {
        "type": "array",
        "items": {
          "type": "object",
          "additionalProperties": false,
          "required": ["path", "line", "message"],
          "properties": {
            "path": { "type": "string" },
            "line": { "type": "integer", "minimum": 0 },
            "message": { "type": "string" }
          }
        }
      }
    }
  }
},
"proof": {
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "required": ["kind", "value", "evidence_path"],
    "properties": {
      "kind": { "type": "string", "minLength": 1 },
```

```
        "value": { "type": "string", "minLength": 1 },
        "evidence_path": { "type": "string", "minLength": 1 }
      }
    },
    "default": []
  },
  "artifacts": {
    "type": "object",
    "additionalProperties": false,
    "required": ["runspec", "runlog", "diff", "snapshot", "changelog"],
    "properties": {
      "runspec": { "type": "string", "minLength": 1 },
      "runlog": { "type": "string", "minLength": 1 },
      "diff": { "type": "string" },
      "snapshot": { "type": "string" },
      "changelog": { "type": "string" }
    }
  }
 }
}
```

Acceptance criteria

- File exists at `maintenance/schemas/report.schema.json`
- Valid JSON
- Requires `results[]` with at least one check
- Requires `artifacts` paths (even if empty strings for optional artifacts)

Implement registries

Implement Registries

Order: **policies.json → targets.json → modes.json**

0.4.1 Create `maintenance/registry/policies.json` (LOCKED v1)

```
{
 "version": "1.0",
 "defaults": {
   "stop_on_fail": true,
   "allow_network": false
 },
 "overrides": {
   "allow_runspec_override_constraints": true,
   "allow_runspec_override_required_proof": false
 },
 "locked_paths_global": [
   ".git/",
   "maintenance/runs/"
 ],
 "write_rules": {
   "when_allow_writes_false": {
     "allowed_write_roots": [
       "maintenance/runs/"
     ]
   }
 }
}
```

Acceptance criteria

- File exists at `maintenance/registry/policies.json`
- JSON valid
- Global locks include `.git/` and `maintenance/runs/`
- `allow_runspec_override_required_proof` is **false**

Create `maintenance/registry/targets.json` (LOCKED v1)

This defines at least one target: **axion-core**. Adjust the `expected_paths` to match your repo later, but keep the structure.

```
{
 "version": "1.0",
 "targets": [
   {
     "target_id": "axion-core",
     "root": ".",
     "type": "axion_core",
     "runtime_contract": {
       "start": "npm run dev",
       "verify": "npm test",
       "build": "npm run build"
     },
     "capabilities_hint": ["node", "npm", "git"],
     "expected_paths": ["axion", "client", "server"]
   }
 ]
}
```

Acceptance criteria

- File exists at `maintenance/registry/targets.json`
- Includes `target_id: "axion-core"`
- Has `runtime_contract.start|verify|build`

Create `maintenance/registry/modes.json` (LOCKED v1)

This file must include **all MM-01..MM-21**. For now, we'll implement **MM-01** as real and set the rest to **stub runners** (they'll fail cleanly with "NOT_IMPLEMENTED" but still write standard artifacts once the dispatcher exists).

Save this JSON:

{
 "version": "1.0",
 "modes": [
  {
   "mode_id": "MM-01",
   "name": "Health Check",
   "description": "Quick status: schemas and registries load; basic target checks.",
   "runner": { "entrypoint": "runners/mm-01-health-check.js", "kind": "node" },
   "allowed_target_types": ["axion_core", "ui", "templates", "other"],
   "required_capabilities": [],
   "default_constraints": {
    "stop_on_fail": true,
    "allow_writes": false,
    "allow_network": false,
    "locked_paths": []
   },
   "required_proof": [{ "kind": "health_check_pass", "value": "maintenance:mm-01" }],
   "outputs": { "diff": false, "snapshot": false, "changelog": false }
  },

  { "mode_id": "MM-02", "name": "Full Verification Run", "description": "Run full verification pipeline.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "command_pass", "value": "verify" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },
  { "mode_id": "MM-03", "name": "Coverage Audit", "description": "Compute coverage deltas.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "coverage_report_generated", "value": "coverage" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },
  { "mode_id": "MM-04", "name": "Drift Detection", "description": "Compare against baseline/snapshot and flag drift.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "diff_generated", "value": "drift" }], "outputs": { "diff": true, "snapshot": false, "changelog": false } },

  { "mode_id": "MM-05", "name": "Planned Upgrade", "description": "Controlled upgrade with dependency closure + proof.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false,

"locked_paths": [] }, "required_proof": [{ "kind": "verify_pass", "value": "post-upgrade" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-06", "name": "Hotfix", "description": "Minimal patch with strict proof.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "verify_pass", "value": "post-hotfix" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-07", "name": "Deprecation Rollout", "description": "Mark deprecated items + warnings + timeline.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "deprecation_applied", "value": "deprecations" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-08", "name": "Breaking Upgrade (Major Version)", "description": "Intentional contract breaks with migration.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "migration_validated", "value": "major-upgrade" }], "outputs": { "diff": true, "snapshot": true, "changelog": true } },

{ "mode_id": "MM-09", "name": "Artifact Migration", "description": "Transform artifacts to new formats.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "migration_applied", "value": "artifacts" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-10", "name": "Backcompat Validation", "description": "Validate old artifacts under compatibility rules.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "backcompat_pass", "value": "compat" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },
{ "mode_id": "MM-11", "name": "Data Repair / Salvage", "description": "Repair broken refs/ids/data.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "repair_complete", "value": "salvage" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },

{ "mode_id": "MM-12", "name": "Rebuild Indexes/Registries", "description": "Recompute registries/indexes from filesystem.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true,

"allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "registry_build_pass", "value": "rebuild" }], "outputs": { "diff": true, "snapshot": false, "changelog": false } },
{ "mode_id": "MM-13", "name": "Registry Integrity Cleanup", "description": "Remove orphans/duplicates; normalize.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "templates"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "registry_clean", "value": "cleanup" }], "outputs": { "diff": true, "snapshot": false, "changelog": false } },

{ "mode_id": "MM-14", "name": "Release Packaging", "description": "Build release artifacts + checksums.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core"], "required_capabilities": ["node", "git"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "package_exists", "value": "release" }], "outputs": { "diff": false, "snapshot": true, "changelog": true } },
{ "mode_id": "MM-15", "name": "Rollback / Revert Release", "description": "Restore prior state + verify.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core"], "required_capabilities": ["node", "git"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "rollback_complete", "value": "rollback" }], "outputs": { "diff": false, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-16", "name": "Snapshot / Archive", "description": "Create snapshot bundle of defined paths.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "snapshot_exists", "value": "snapshot" }], "outputs": { "diff": false, "snapshot": true, "changelog": false } },

{ "mode_id": "MM-17", "name": "Dependency Refresh", "description": "Update deps/lockfiles + verify.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui"], "required_capabilities": ["node", "npm"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": true, "locked_paths": [] }, "required_proof": [{ "kind": "deps_updated", "value": "dependencies" }], "outputs": { "diff": true, "snapshot": false, "changelog": true } },
{ "mode_id": "MM-18", "name": "Workspace Compatibility Check", "description": "Validate runtime contract/entrypoints.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "workspace_contract_pass", "value": "workspace" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },
{ "mode_id": "MM-19", "name": "CI Sanity Run", "description": "Run CI-equivalent pipeline.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" },

"allowed_target_types": ["axion_core", "ui"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "ci_pass", "value": "ci" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },

{ "mode_id": "MM-20", "name": "Failure Triage Mode", "description": "Reproduce, isolate, capture failing case.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": false, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "triage_report_generated", "value": "triage" }], "outputs": { "diff": false, "snapshot": false, "changelog": false } },
{ "mode_id": "MM-21", "name": "Emergency Recovery Mode", "description": "Restore to stable/verifiable state.", "runner": { "entrypoint": "runners/mm-stub-not-implemented.js", "kind": "node" }, "allowed_target_types": ["axion_core", "ui", "templates", "other"], "required_capabilities": ["node"], "default_constraints": { "stop_on_fail": true, "allow_writes": true, "allow_network": false, "locked_paths": [] }, "required_proof": [{ "kind": "recovery_actions_applied", "value": "recovery" }], "outputs": { "diff": true, "snapshot": true, "changelog": false } }
]
}

Acceptance criteria

- File exists at `maintenance/registry/modes.json`
- Contains **exactly one** entry per MM-01..MM-21
- MM-01 points to `runners/mm-01-health-check.js`
- All others point to `runners/mm-stub-not-implemented.js` for now

Add the stub runner + dispatcher

Add the stub runner + dispatcher (Node/JS, v1)

You'll create **two files**:

1. `maintenance/runners/mm-stub-not-implemented.js`

2. `maintenance/runners/dispatcher.js`

These are enough to run any mode and always get `runspec/runlog/report`, even if the mode isn't implemented yet.

---

1) `maintenance/runners/mm-stub-not-implemented.js`

```
// maintenance/runners/mm-stub-not-implemented.js
'use strict';

const fs = require('fs');
const path = require('path');

function nowIso() {
 return new Date().toISOString();
}

function safeWrite(filePath, content) {
 fs.mkdirSync(path.dirname(filePath), { recursive: true });
 fs.writeFileSync(filePath, content, 'utf8');
}

function appendLog(runlogPath, line) {
 fs.mkdirSync(path.dirname(runlogPath), { recursive: true });
 fs.appendFileSync(runlogPath, `${line}\n`, 'utf8');
}
```

```
function main() {
 const {
   MODE_ID,
   RUN_ID,
   RUN_DIR,
   TARGET_ID,
   TARGET_ROOT
 } = process.env;

 const runDir = RUN_DIR || '';
 if (!runDir) {
   console.error('mm-stub-not-implemented: missing RUN_DIR');
   process.exit(1);
 }

 const runlogPath = path.join(runDir, 'runlog.md');
 const reportPath = path.join(runDir, 'report.json');

 appendLog(runlogPath, `# Run Log`);
 appendLog(runlogPath, `- run_id: ${RUN_ID}`);
 appendLog(runlogPath, `- mode_id: ${MODE_ID}`);
 appendLog(runlogPath, `- target_id: ${TARGET_ID}`);
 appendLog(runlogPath, `- target_root: ${TARGET_ROOT}`);
 appendLog(runlogPath, ``);
 appendLog(runlogPath, `## Result`);
 appendLog(runlogPath, `NOT_IMPLEMENTED`);

 const started = nowIso();
 const finished = nowIso();

 const report = {
   run_id: RUN_ID || '',
   mode_id: MODE_ID || '',
   status: 'FAIL',
   target: {
     target_id: TARGET_ID || '',
     commit: '',
     branch: ''
   },
   started_at: started,
   finished_at: finished,
   results: [
     {
       check_id: 'implemented',
```

```
      status: 'FAIL',
      summary: 'Mode runner not implemented.',
      pointers: [
        { path: 'maintenance/runners/mm-stub-not-implemented.js', line: 0, message: 'Replace
stub with real runner.' }
      ]
    }
  ],
  proof: [],
  artifacts: {
    runspec: 'runspec.json',
    runlog: 'runlog.md',
    diff: '',
    snapshot: '',
    changelog: ''
  }
};

safeWrite(reportPath, JSON.stringify(report, null, 2));

process.exit(1);
}

main();
```

Acceptance criteria

- If called with env vars, it writes:
  - `runlog.md`
  - `report.json` (schema-valid)
- Exits with code **1**

---

2) `maintenance/runners/dispatcher.js` (v1)

```
// maintenance/runners/dispatcher.js
'use strict';

const fs = require('fs');
const path = require('path');
const { spawnSync } = require('child_process');

// ---------- helpers ----------
function readJson(filePath) {
 return JSON.parse(fs.readFileSync(filePath, 'utf8'));
}

function safeWrite(filePath, content) {
 fs.mkdirSync(path.dirname(filePath), { recursive: true });
 fs.writeFileSync(filePath, content, 'utf8');
}

function appendLog(runlogPath, line) {
 fs.mkdirSync(path.dirname(runlogPath), { recursive: true });
 fs.appendFileSync(runlogPath, `${line}\n`, 'utf8');
}

function nowIso() {
 return new Date().toISOString();
}

function makeRunId(modeId, envTag) {
 // YYYYMMDD_HHMMSS_<env>_<mode>_<shortid>
 const d = new Date();
 const pad = (n) => String(n).padStart(2, '0');
 const yyyy = d.getFullYear();
 const mm = pad(d.getMonth() + 1);
 const dd = pad(d.getDate());
 const hh = pad(d.getHours());
 const mi = pad(d.getMinutes());
 const ss = pad(d.getSeconds());
 const ts = `${yyyy}${mm}${dd}_${hh}${mi}${ss}`;
```

```
 const shortid = Math.random().toString(16).slice(2, 10);
 return `${ts}_${envTag}_${modeId}_${shortid}`;
}

function exists(p) {
 try { fs.accessSync(p); return true; } catch { return false; }
}

function requireArg(flag) {
 const idx = process.argv.indexOf(flag);
 if (idx === -1) return null;
 return process.argv[idx + 1] || null;
}

function failEarly(runDir, runId, modeId, targetId, reason) {
 const runlogPath = path.join(runDir, 'runlog.md');
 const reportPath = path.join(runDir, 'report.json');

 appendLog(runlogPath, `# Run Log`);
 appendLog(runlogPath, `- run_id: ${runId}`);
 appendLog(runlogPath, `- mode_id: ${modeId}`);
 appendLog(runlogPath, `- target_id: ${targetId}`);
 appendLog(runlogPath, ``);
 appendLog(runlogPath, `## Failure`);
 appendLog(runlogPath, reason);

 const t = nowIso();
 const report = {
   run_id: runId,
   mode_id: modeId,
   status: 'FAIL',
   target: { target_id: targetId, commit: '', branch: '' },
   started_at: t,
   finished_at: t,
   results: [
     {
       check_id: 'dispatcher',
       status: 'FAIL',
       summary: reason,
       pointers: []
     }
   ],
   proof: [],
   artifacts: {
```

```
    runspec: 'runspec.json',
    runlog: 'runlog.md',
    diff: '',
    snapshot: '',
    changelog: ''
  }
};

safeWrite(reportPath, JSON.stringify(report, null, 2));
return 1;
}

// Very light schema checks (MVP). Full JSON-schema validation can be added later.
function validateRunspecShape(spec) {
 const requiredTop = ['run_id', 'mode_id', 'target', 'scope', 'constraints', 'requested_outputs',
'expected_proof', 'metadata'];
 for (const k of requiredTop) {
   if (!(k in spec)) return `runspec missing required field: ${k}`;
 }
 if (!/^MM-(0[1-9]|1[0-9]|2[0-1])$/.test(spec.mode_id)) return `invalid mode_id: ${spec.mode_id}`;
 if (!spec.target || typeof spec.target !== 'object') return `runspec.target must be object`;
 if (!spec.target.target_id) return `runspec.target.target_id required`;
 if (!spec.target.root) return `runspec.target.root required`;
 if (!spec.target.type) return `runspec.target.type required`;
 if (!spec.constraints || typeof spec.constraints !== 'object') return `runspec.constraints must be
object`;
 if (!Array.isArray(spec.expected_proof) || spec.expected_proof.length < 1) return
`runspec.expected_proof must be non-empty array`;
 return null;
}

function checkCapabilities(required) {
 // MVP: support "node", "npm", "git"
 // We only check that commands exist in PATH.
 const missing = [];
 const cmdFor = (cap) => {
   if (cap === 'node') return 'node';
   if (cap === 'npm') return 'npm';
   if (cap === 'git') return 'git';
   return null;
 };
 for (const cap of required) {
   const cmd = cmdFor(cap);
   if (!cmd) continue;
```

```
  const res = spawnSync(cmd, ['--version'], { stdio: 'ignore' });
  if (res.status !== 0) missing.push(cap);
 }
 return missing;
}

// ---------- main ----------
function main() {
 const modeId = process.argv[2];
 if (!modeId || !/^MM-(0[1-9]|1[0-9]|2[0-1])$/.test(modeId)) {
  console.error('Usage: node maintenance/runners/dispatcher.js <MM-xx> --spec <path>');
  process.exit(2);
 }

 const specPathArg = requireArg('--spec');
 if (!specPathArg) {
  console.error('Missing --spec <path>');
  process.exit(2);
 }

 const repoRoot = process.cwd();
 const maintenanceRoot = path.join(repoRoot, 'maintenance');
 const policiesPath = path.join(maintenanceRoot, 'registry', 'policies.json');
 const targetsPath = path.join(maintenanceRoot, 'registry', 'targets.json');
 const modesPath = path.join(maintenanceRoot, 'registry', 'modes.json');

 if (!exists(policiesPath) || !exists(targetsPath) || !exists(modesPath)) {
  console.error('Missing maintenance registries. Ensure maintenance/registry/*.json exists.');
  process.exit(2);
 }

 const policies = readJson(policiesPath);
 const targets = readJson(targetsPath);
 const modes = readJson(modesPath);

 const modeDef = (modes.modes || []).find((m) => m.mode_id === modeId);
 if (!modeDef) {
  console.error(`Mode not found in modes.json: ${modeId}`);
  process.exit(2);
 }

 // Load runspec
 const specAbs = path.isAbsolute(specPathArg) ? specPathArg : path.join(repoRoot,
specPathArg);
```

```
if (!exists(specAbs)) {
  console.error(`Spec file not found: ${specAbs}`);
  process.exit(2);
}

const runspec = readJson(specAbs);

// Generate/normalize run_id (dispatcher is authority)
const envTag = (runspec.metadata && runspec.metadata.env) ? runspec.metadata.env : 'local';
const runId = makeRunId(modeId, envTag);
runspec.run_id = runId; // overwrite to avoid collisions / user error
runspec.mode_id = modeId; // enforce

const shapeErr = validateRunspecShape(runspec);
const runsDir = path.join(maintenanceRoot, 'runs');
const runDir = path.join(runsDir, runId);

fs.mkdirSync(runDir, { recursive: true });

// Write finalized runspec into run folder (source of truth)
safeWrite(path.join(runDir, 'runspec.json'), JSON.stringify(runspec, null, 2));

// Basic log header
const runlogPath = path.join(runDir, 'runlog.md');
appendLog(runlogPath, `# Run Log`);
appendLog(runlogPath, `- run_id: ${runId}`);
appendLog(runlogPath, `- mode_id: ${modeId}`);
appendLog(runlogPath, `- started_at: ${nowIso()}`);
appendLog(runlogPath, ``);

if (shapeErr) {
  const code = failEarly(runDir, runId, modeId, (runspec.target && runspec.target.target_id) || '',
shapeErr);
  process.exit(code);
}

// Resolve target from targets.json (authoritative)
const targetId = runspec.target.target_id;
const targetDef = (targets.targets || []).find((t) => t.target_id === targetId);
if (!targetDef) {
  const code = failEarly(runDir, runId, modeId, targetId, `Target not found in targets.json:
${targetId}`);
  process.exit(code);
}
```

```
 // Capabilities check (mode required + hinted target capabilities are not enforced here)
 const requiredCaps = modeDef.required_capabilities || [];
 const missingCaps = checkCapabilities(requiredCaps);
 if (missingCaps.length) {
   const code = failEarly(runDir, runId, modeId, targetId, `Missing required capabilities:
${missingCaps.join(', ')}`);
   process.exit(code);
 }

 // Merge constraints: policies.defaults -> mode.default_constraints -> runspec.constraints (if
allowed)
 const mergedConstraints = {
   stop_on_fail: (policies.defaults && policies.defaults.stop_on_fail) ?? true,
   allow_network: (policies.defaults && policies.defaults.allow_network) ?? false,
   allow_writes: false,
   locked_paths: []
 };

 if (modeDef.default_constraints) {
   Object.assign(mergedConstraints, modeDef.default_constraints);
 }

 const allowOverrideConstraints = !!(policies.overrides &&
policies.overrides.allow_runspec_override_constraints);
 if (allowOverrideConstraints && runspec.constraints) {
   Object.assign(mergedConstraints, runspec.constraints);
 }

 runspec.constraints = mergedConstraints;
 safeWrite(path.join(runDir, 'runspec.json'), JSON.stringify(runspec, null, 2));

 // REQUIRED PROOF: cannot be overridden (v1 policy), but we record both for transparency.
 const requiredProof = modeDef.required_proof || [];
 const allowOverrideProof = !!(policies.overrides &&
policies.overrides.allow_runspec_override_required_proof);
 if (!allowOverrideProof) {
   runspec.expected_proof = requiredProof;
   safeWrite(path.join(runDir, 'runspec.json'), JSON.stringify(runspec, null, 2));
 }

 // Compute runner absolute path
 const runnerRel = modeDef.runner && modeDef.runner.entrypoint;
 if (!runnerRel) {
```

```
  const code = failEarly(runDir, runId, modeId, targetId, `Mode runner entrypoint missing for
${modeId}`);
  process.exit(code);
}

const runnerAbs = path.join(maintenanceRoot, runnerRel);
if (!exists(runnerAbs)) {
  const code = failEarly(runDir, runId, modeId, targetId, `Runner file not found: ${runnerRel}`);
  process.exit(code);
}

// Execute runner (Node)
appendLog(runlogPath, `## Runner`);
appendLog(runlogPath, `- runner: ${runnerRel}`);
appendLog(runlogPath, ``);

const targetRootAbs = path.isAbsolute(targetDef.root) ? targetDef.root : path.join(repoRoot,
targetDef.root);

const childEnv = {
  ...process.env,
  MODE_ID: modeId,
  RUN_ID: runId,
  RUN_DIR: runDir,
  TARGET_ID: targetId,
  TARGET_ROOT: targetRootAbs
};

const startedAt = nowIso();
const res = spawnSync('node', [runnerAbs], { env: childEnv, cwd: repoRoot, encoding: 'utf8' });

// Append runner output to runlog
appendLog(runlogPath, `## Runner Output`);
if (res.stdout) appendLog(runlogPath, res.stdout.trimEnd());
if (res.stderr) appendLog(runlogPath, res.stderr.trimEnd());
appendLog(runlogPath, ``);

// Guarantee report exists
const reportPath = path.join(runDir, 'report.json');
if (!exists(reportPath)) {
  // if runner failed to write report, dispatcher writes minimal FAIL report
  const finishedAt = nowIso();
  const report = {
    run_id: runId,
```

```
    mode_id: modeId,
    status: 'FAIL',
    target: { target_id: targetId, commit: '', branch: '' },
    started_at: startedAt,
    finished_at: finishedAt,
    results: [
      {
        check_id: 'report_written',
        status: 'FAIL',
        summary: 'Runner did not write report.json',
        pointers: [{ path: runnerRel, line: 0, message: 'Runner must always write report.json' }]
      }
    ],
    proof: [],
    artifacts: {
      runspec: 'runspec.json',
      runlog: 'runlog.md',
      diff: '',
      snapshot: '',
      changelog: ''
    }
  };
  safeWrite(reportPath, JSON.stringify(report, null, 2));
}

appendLog(runlogPath, `## Finished`);
appendLog(runlogPath, `- finished_at: ${nowIso()}`);
appendLog(runlogPath, `- runner_exit_code: ${res.status === null ? 'null' : String(res.status)}`);

// Exit code policy: if runner exit 0 => pass; otherwise fail.
// (Later: read report.status for WARN=2, etc.)
process.exit(res.status === 0 ? 0 : 1);
}

main();
```

Acceptance criteria

- Running dispatcher creates a run folder under `maintenance/runs/<run_id>/` containing:
    - `runspec.json` (finalized by dispatcher)

    - `runlog.md`
    - `report.json` (even if runner fails)
- Modes without real runners use the stub runner and fail cleanly.

How to run it (for now)

Create a simple test runspec (we'll formalize later). Example file: `maintenance/tmp.runspec.json`:

```
{
 "run_id": "placeholder",
 "mode_id": "MM-01",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "placeholder", "value": "ignored_by_policy" }],
 "metadata": { "env": "local", "requested_by": "user", "notes": "" }
}
```

Run:

```
node maintenance/runners/dispatcher.js MM-02 --spec maintenance/tmp.runspec.json
```

MM-02 will route to the stub and create artifacts.

MM-01 Health Check Runner

MM-01 Health Check Runner (PASS-capable)

Create: `maintenance/runners/mm-01-health-check.js`

This runner only checks that the maintenance control plane is structurally healthy (schemas + registries exist and parse) and that the target root exists. It writes a schema-valid `report.json` and exits **0 on PASS**, **1 on FAIL**.

```
// maintenance/runners/mm-01-health-check.js
'use strict';

const fs = require('fs');
const path = require('path');

function nowIso() {
 return new Date().toISOString();
}

function exists(p) {
 try { fs.accessSync(p); return true; } catch { return false; }
}

function readJson(p) {
 return JSON.parse(fs.readFileSync(p, 'utf8'));
}

function safeWrite(filePath, content) {
 fs.mkdirSync(path.dirname(filePath), { recursive: true });
 fs.writeFileSync(filePath, content, 'utf8');
}

function appendLog(runlogPath, line) {
 fs.mkdirSync(path.dirname(runlogPath), { recursive: true });
 fs.appendFileSync(runlogPath, `${line}\n`, 'utf8');
}

function getGitMeta(targetRoot) {
 // Best-effort; no hard dependency.
 try {
   const headPath = path.join(targetRoot, '.git', 'HEAD');
   if (!exists(headPath)) return { commit: '', branch: '' };
   const head = fs.readFileSync(headPath, 'utf8').trim();
   if (head.startsWith('ref:')) {
```

```
    const ref = head.replace('ref:', '').trim();
    const refPath = path.join(targetRoot, '.git', ref);
    const branch = ref.split('/').slice(-1)[0] || '';
    const commit = exists(refPath) ? fs.readFileSync(refPath, 'utf8').trim() : '';
    return { commit, branch };
  }
  // detached HEAD
  return { commit: head, branch: '' };
} catch {
  return { commit: '', branch: '' };
}
}

function main() {
 const {
   MODE_ID,
   RUN_ID,
   RUN_DIR,
   TARGET_ID,
   TARGET_ROOT
 } = process.env;

 const startedAt = nowIso();

 if (!RUN_DIR) {
   console.error('MM-01: missing RUN_DIR');
   process.exit(1);
 }

 const runlogPath = path.join(RUN_DIR, 'runlog.md');
 const reportPath = path.join(RUN_DIR, 'report.json');

 appendLog(runlogPath, `# Run Log`);
 appendLog(runlogPath, `- run_id: ${RUN_ID}`);
 appendLog(runlogPath, `- mode_id: ${MODE_ID}`);
 appendLog(runlogPath, `- target_id: ${TARGET_ID}`);
 appendLog(runlogPath, `- target_root: ${TARGET_ROOT}`);
 appendLog(runlogPath, `- started_at: ${startedAt}`);
 appendLog(runlogPath, ``);

 const repoRoot = process.cwd();
 const maintenanceRoot = path.join(repoRoot, 'maintenance');

 const checks = [];
```

```
const pointers = [];

function addCheck(check_id, ok, summary, ptrs = []) {
  checks.push({
    check_id,
    status: ok ? 'PASS' : 'FAIL',
    summary,
    pointers: ptrs
  });
  if (!ok) pointers.push(...ptrs);
}

// 1) maintenance skeleton exists
const requiredPaths = [
  'maintenance/registry',
  'maintenance/schemas',
  'maintenance/runners',
  'maintenance/runbooks',
  'maintenance/prompts'
];
for (const rel of requiredPaths) {
  const abs = path.join(repoRoot, rel);
  addCheck(
    `exists:${rel}`,
    exists(abs),
    exists(abs) ? 'OK' : 'Missing required folder.',
    exists(abs) ? [] : [{ path: rel, line: 0, message: 'Create the required folder.' }]
  );
}

// 2) schemas exist and parse as JSON
const schemaFiles = [
  'maintenance/schemas/runspec.schema.json',
  'maintenance/schemas/report.schema.json'
];
for (const rel of schemaFiles) {
  const abs = path.join(repoRoot, rel);
  if (!exists(abs)) {
    addCheck(`schema:${rel}`, false, 'Schema file missing.', [{ path: rel, line: 0, message: 'Add
the schema file.' }]);
    continue;
  }
  try {
    readJson(abs);
```

```
    addCheck(`schema:${rel}`, true, 'Schema parses as JSON.', []);
  } catch (e) {
    addCheck(`schema:${rel}`, false, `Schema JSON parse failed: ${String(e.message || e)}`, [{
path: rel, line: 0, message: 'Fix JSON formatting.' }]);
  }
}

// 3) registries exist and parse as JSON
const registryFiles = [
  'maintenance/registry/policies.json',
  'maintenance/registry/targets.json',
  'maintenance/registry/modes.json'
];
for (const rel of registryFiles) {
  const abs = path.join(repoRoot, rel);
  if (!exists(abs)) {
    addCheck(`registry:${rel}`, false, 'Registry file missing.', [{ path: rel, line: 0, message: 'Add
the registry file.' }]);
    continue;
  }
  try {
    readJson(abs);
    addCheck(`registry:${rel}`, true, 'Registry parses as JSON.', []);
  } catch (e) {
    addCheck(`registry:${rel}`, false, `Registry JSON parse failed: ${String(e.message || e)}`, [{
path: rel, line: 0, message: 'Fix JSON formatting.' }]);
  }
}

// 4) target root exists
const targetOk = TARGET_ROOT && exists(TARGET_ROOT);
addCheck(
  'target_root',
  !!targetOk,
  targetOk ? 'Target root exists.' : 'Target root missing or not accessible.',
  targetOk ? [] : [{ path: TARGET_ROOT || '', line: 0, message: 'Check target root path and
permissions.' }]
);

const finishedAt = nowIso();
const { commit, branch } = getGitMeta(TARGET_ROOT || repoRoot);

// overall status
const anyFail = checks.some((c) => c.status === 'FAIL');
```

```
 const status = anyFail ? 'FAIL' : 'PASS';

 appendLog(runlogPath, `## Checks`);
 for (const c of checks) {
   appendLog(runlogPath, `- [${c.status}] ${c.check_id}: ${c.summary}`);
 }
 appendLog(runlogPath, ``);
 appendLog(runlogPath, `## Result`);
 appendLog(runlogPath, status);
 appendLog(runlogPath, `- finished_at: ${finishedAt}`);

 const report = {
   run_id: RUN_ID || '',
   mode_id: MODE_ID || '',
   status,
   target: {
     target_id: TARGET_ID || '',
     commit,
     branch
   },
   started_at: startedAt,
   finished_at: finishedAt,
   results: checks,
   proof: status === 'PASS'
     ? [{ kind: 'health_check_pass', value: 'maintenance:mm-01', evidence_path: 'runlog.md' }]
     : [],
   artifacts: {
     runspec: 'runspec.json',
     runlog: 'runlog.md',
     diff: '',
     snapshot: '',
     changelog: ''
   }
 };

 safeWrite(reportPath, JSON.stringify(report, null, 2));
 process.exit(status === 'PASS' ? 0 : 1);
}

main();
```

Quick smoke run (MM-01)

Use the same `maintenance/tmp.runspec.json` from earlier (target_id `axion-core`, env `local`).

Command:

node maintenance/runners/dispatcher.js MM-01 --spec maintenance/tmp.runspec.json

Expected:

- exit code `0` if everything exists/valid
- new folder: `maintenance/runs/<run_id>/` with `runspec.json`, `runlog.md`, `report.json`

MVP

MVP `allow_writes=false` enforcement (git diff)

We'll add a **post-run check** in `maintenance/runners/dispatcher.js`:

- If `runspec.constraints.allow_writes === false`
- then after the runner finishes, we verify **no repo changes occurred** outside allowed write roots (which, in v1, is only `maintenance/runs/`).

What this prevents

Even if an agent/runner tries to edit source files during a read-only mode, the run will be marked **FAIL**.

Patch to apply to `maintenance/runners/dispatcher.js`

1) Add this helper near the other helpers

```
function gitStatusPorcelain(repoRoot) {
 const res = spawnSync('git', ['status', '--porcelain'], { cwd: repoRoot, encoding: 'utf8' });
 if (res.status !== 0) return { ok: false, output: (res.stderr || res.stdout || '').trim() };
 return { ok: true, output: (res.stdout || '').trim() };
}

function hasDisallowedChanges(porcelainOutput, allowedWriteRoots) {
 if (!porcelainOutput) return { changed: false, disallowed: [] };
 const lines = porcelainOutput.split('\n').map(l => l.trim()).filter(Boolean);

 // porcelain format begins with "XY <path>" (or similar); path starts at position 3.
 const paths = lines.map(l => l.slice(3).trim()).filter(Boolean);

 const disallowed = [];
 for (const p of paths) {
   const normalized = p.replace(/\\/g, '/');
   const allowed = allowedWriteRoots.some(root => normalized.startsWith(root));
   if (!allowed) disallowed.push(normalized);
```

```
 }

 return { changed: paths.length > 0, disallowed };
}
```

2) Capture “before” git status right before executing the runner

Find the spot just before:

```
const res = spawnSync('node', [runnerAbs], ...
```

Insert this above it:

```
let gitBefore = null;
const allowWrites = !!runspec.constraints.allow_writes;

const needsGitEnforcement = !allowWrites;
if (needsGitEnforcement) {
 const st = gitStatusPorcelain(repoRoot);
 if (!st.ok) {
   // If git isn't available, we can't enforce via git in MVP.
   // We log a WARN but continue (you can tighten later).
   appendLog(runlogPath, `## Enforcement`);
   appendLog(runlogPath, `WARN: git status check failed; cannot enforce allow_writes=false via
git. ${st.output}`);
 } else {
   gitBefore = st.output;
 }
}
```

3) After runner completes and report is guaranteed, enforce

Insert this **after** the `Guarantee report exists` block and **before** the final `process.exit(...)`:

```
if (needsGitEnforcement && gitBefore !== null) {
 const stAfter = gitStatusPorcelain(repoRoot);
 appendLog(runlogPath, `## Enforcement`);
 if (!stAfter.ok) {
   appendLog(runlogPath, `WARN: git status after-run failed; cannot enforce allow_writes=false.
${stAfter.output}`);
 } else {
   const allowedRoots = ['maintenance/runs/']; // v1 policy: only maintenance outputs may
change
   const { disallowed } = hasDisallowedChanges(stAfter.output, allowedRoots);
```

```
  if (disallowed.length > 0) {
    appendLog(runlogPath, `FAIL: Disallowed changes detected while allow_writes=false:`);
    for (const p of disallowed) appendLog(runlogPath, `- ${p}`);

    // Overwrite/force report to FAIL (source of truth)
    const reportPath = path.join(runDir, 'report.json');
    const report = readJson(reportPath);
    report.status = 'FAIL';

    report.results.push({
      check_id: 'allow_writes_enforcement',
      status: 'FAIL',
      summary: `Disallowed repo changes detected while allow_writes=false.`,
      pointers: disallowed.map(p => ({ path: p, line: 0, message: 'Revert changes or run with
allow_writes=true in an appropriate mode.' }))
    });

    safeWrite(reportPath, JSON.stringify(report, null, 2));

    // Force failure exit code
    process.exit(1);
  } else {
    appendLog(runlogPath, `PASS: No disallowed changes detected (allow_writes=false).`);
  }
 }
}
```

Acceptance criteria (Foundation enforcement)

- For a run with `allow_writes=false`, if any file outside `maintenance/runs/` is modified, the dispatcher:
    - sets `report.status=FAIL`
    - adds `allow_writes_enforcement` check to `report.results`
    - exits code `1`

Core Operations

Core Operations (Documentation Outline)

P1.0 Phase Goal

Implement the “operational spine” modes so you can:

- verify Axion
- rebuild indexes/registries
- confirm workspace/runtime contract
- triage failures
- recover to a stable state

P1.1 Modes to Implement (in this exact order)

1. **MM-02 — Full Verification Run**
2. **MM-18 — Workspace Compatibility Check**
3. **MM-12 — Rebuild Indexes/Registries**
4. **MM-20 — Failure Triage Mode**
5. **MM-21 — Emergency Recovery Mode**

P1.2 What “implemented” means (Definition of Done per mode)

A mode is considered implemented only if all 4 are true:

- **Registry entry** exists in `maintenance/registry/modes.json` and points to a real runner (not the stub)
- **Runner file** exists in `maintenance/runners/`
- **Runbook file** exists in `maintenance/runbooks/`

- A run produces:
    - `runspec.json`, `runlog.md`, `report.json`
    - `report.json.status` is PASS/FAIL/WARN
    - `report.proof[]` includes the mode's required proof kinds

P1.3 Shared runner pattern (all Phase 1 modes use this)

Each runner must implement:

- Preflight: load target runtime_contract, check capabilities
- Execute: run mode steps
- Evidence capture: summarize outputs + exit codes
- Report: write schema-valid `report.json` + pointers
    -

MM-02 Full Verification Run

P1.1.1 — MM-02 Full Verification Run (Mode Spec)

Mode ID

MM-02

Name

Full Verification Run

Purpose

Run the target workspace's **full verification pipeline** (tests/gates/verify command) and emit a **machine-verifiable report** with pointers on failure.

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`
(Practical default: start with `axion_core`.)

Required Capabilities

- `node`
- `npm`
- `git` *(optional but recommended for commit/branch metadata and enforcement checks)*

Runspec Inputs Used

- `target.target_id` (resolved via `targets.json`)
- `target.root` (resolved via `targets.json`)
- `scope` *(v1: informational only; runner runs full verify unless you later implement scope-aware test selection)*
- `constraints.stop_on_fail`

- `constraints.allow_writes` (must be `false` for MM-02 in v1)
- `constraints.allow_network` (default `false`)
- `metadata.env`, `metadata.notes`

Runtime Contract Dependency

MM-02 executes the verification command from:

- `maintenance/registry/targets.json`
    - `targets[].runtime_contract.verify`

Example:

- `npm test`

Steps (Deterministic)

1. **Preflight**
    - Confirm target exists
    - Confirm required capabilities (`node`, `npm`)
    - Record git metadata if available (branch/commit)
2. **Execute Verify**
    - Run: `runtime_contract.verify`
    - Capture:
        - exit code
        - stdout/stderr (trimmed summary)
        - duration

3. **Write Outputs**
    - Append command output summary to `runlog.md`
    - Write `report.json`:
        - PASS if exit code = 0
        - FAIL otherwise
        - include pointers (best-effort) if output mentions file/line (optional v1)

Proof Requirements (Locked)

`report.proof[]` must include at least:

- `{ kind: "command_pass", value: "<verify command>", evidence_path: "runlog.md" }` on PASS

On FAIL, proof can be empty, but `results[]` must include the failing check.

Required `report.results[]` entries (minimum)

- `check_id: "verify"`
- `status: PASS|FAIL`
- `summary: "<short outcome>"`
- `pointers: []` (or best-effort parsed pointers)

Outputs

Always produced:

- `maintenance/runs/<run_id>/runspec.json`
- `maintenance/runs/<run_id>/runlog.md`

- `maintenance/runs/<run_id>/report.json`

Optional:

- none (MM-02 is read-only by default)

Failure Semantics

- If verify command fails:
  - `report.status = FAIL`
  - `results[0].status = FAIL`
  - exit code should be `1` (dispatcher enforces)

Mode Registry Entry (to switch from stub to real)

Update in `maintenance/registry/modes.json`:

- Change runner from:
  - `runners/mm-stub-not-implemented.js`
- To:
  - `runners/mm-02-full-verify.js`

Runner File Name

`maintenance/runners/mm-02-full-verify.js`

Runbook File Name

`maintenance/runbooks/MM-02.full-verify.md`

Example runspec (MM-02)

{

"run_id": "placeholder",
"mode_id": "MM-02",
"target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
"scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network": false },
"requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": { "env": "local", "requested_by": "agent", "notes": "Full verify run" }
}

MM-18 Workspace Compatibility Check

P1.1.2 — MM-18 Workspace Compatibility Check (Mode Spec)

Mode ID

MM-18

Name

Workspace Compatibility Check

Purpose

Validate the **workspace/runtime contract** so the target can boot reliably (Replit-safe / CI-safe). This is a **static + light dynamic** check designed to catch “workspace breakers” early.

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`

Required Capabilities

- `node` *(for parsing JSON + running checks if implemented in Node)*
- `npm` *(optional; only needed if you execute any scripts)*
- `git` *(optional; metadata only)*

Runspec Inputs Used

- `target.target_id` (resolved via `targets.json`)
- `constraints.allow_writes` (must be `false` for MM-18 in v1)
- `metadata.env`, `metadata.notes`

Checks Performed (Locked v1 set)

MM-18 produces a report containing these checks (PASS/FAIL per check):

Check A — Target root exists

- Verify `TARGET_ROOT` exists and is accessible.

Check B — Runtime contract exists in targets registry

- Confirm `targets.json` has `runtime_contract.start|verify|build` for `target_id`.

Check C — Node project basics (if `package.json` exists)

- `package.json` exists and parses
- `scripts` contains the commands referenced by runtime contract where applicable:
  - If runtime contract uses `npm run <script>`, `<script>` must exist
- Optional:
  - `package-lock.json`/`pnpm-lock.yaml`/`yarn.lock` presence check (WARN only)

Check D — Replit contract (if `.replit` exists)

- `.replit` file exists and is not empty
- (v1) basic parse checks:
  - contains a `run` command line OR expected minimal structure
- If `.replit` run command references a file path, verify that file exists
  - Example: `node server/index.js` → check `server/index.js` exists

Check E — Entry points exist (best-effort)

- If runtime contract is:
  - `npm run dev` → ensure script exists

    - `node <file>` → ensure file exists

- For TS repos:
    - if start references `ts-node`/`tsx`/`vite`, check config files exist (WARN if missing)
        - `tsconfig.json`, `vite.config.*` (if relevant)

Check F — Port binding expectation (static)

- Confirm the backend entrypoint uses `process.env.PORT` or documented port strategy (best-effort grep)
- If not detectable, emit WARN (not FAIL) in v1.

> Note: MM-18 should be conservative: prefer WARN over FAIL when detection is uncertain.

Proof Requirements (Locked)

On PASS, `report.proof[]` must include:

- `{ kind: "workspace_contract_pass", value: "workspace", evidence_path: "runlog.md" }`

Required `report.results[]` entries (minimum)

At least these `check_id`s (exact IDs recommended):

- `target_root`
- `runtime_contract`
- `package_json`
- `scripts_present`
- `replit_contract`

- `entrypoints_exist`
- `port_strategy`

Each must include:

- status
- summary
- pointers (on FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Optional:

- none (read-only)

Failure Semantics

- Any FAIL check → `report.status = FAIL`
- WARN checks do not fail the mode unless you later enable strict policy.

Mode Registry Entry (switch stub to real)

Update `maintenance/registry/modes.json` runner:

- from `runners/mm-stub-not-implemented.js`

- to `runners/mm-18-workspace-compat.js`

Runner File Name

`maintenance/runners/mm-18-workspace-compat.js`

Runbook File Name

`maintenance/runbooks/MM-18.workspace-compat.md`

Example runspec (MM-18)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-18",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Workspace contract check" }
}
```

MM-12 Rebuild Indexes/Registries

P1.1.3 — MM-12 Rebuild Indexes/Registries (Mode Spec)

Mode ID

MM-12

Name

Rebuild Indexes/Registries

Purpose

Recompute and/or regenerate Axion's **indexes/registries/manifests** from source-of-truth inputs (templates/artifacts/filesystem) to restore consistency after changes, drift, or corruption.

This mode is intended to be the **fastest way to "make the catalogs true again."**

Applies To

`allowed_target_types`: `axion_core | templates`
(Primary: `axion_core`.)

Required Capabilities

- `node`
- `npm` *(if rebuild command is an npm script)*
- `git` *(recommended for diff capture + enforcement)*

Runspec Inputs Used

- `target.target_id` (resolved via `targets.json`)
- `scope` (v1: informational; v2 can rebuild only impacted paths/components)
- `constraints.allow_writes` (**must be true** for MM-12 because rebuild writes files)
- `requested_outputs.diff` (recommended true: capture what changed)

- `metadata.env`, `metadata.notes`

Runtime Contract Dependency

MM-12 needs a rebuild command. In v1, define it in one of two ways:

Option A (preferred): targets.json
Add to `targets[].runtime_contract`:

- `rebuild_indexes`: `"npm run axion:rebuild"` (example)

Option B: mode runner defaults
Runner has a default command list for axion-core if the target doesn't provide one.

> Documentation rule: MM-12 must not "guess" silently. If `rebuild_indexes` is not defined and no default exists, mode FAILs with clear pointers.

Steps (Deterministic)

1. **Preflight**
    - Confirm target exists
    - Confirm capabilities
    - Confirm `allow_writes=true`
    - Determine rebuild command(s):
        - `runtime_contract.rebuild_indexes` if present
        - else runner default (if configured)
    - Record git metadata and "before" status for diff (if git available)
2. **Execute Rebuild**
    - Run rebuild command(s)
    - Capture stdout/stderr and exit codes

3. **Post-Rebuild Integrity Check**
    - Validate that key registry/index files exist and parse (best-effort):
        - Example targets (adjust to your repo):
            - `axion/registry/*`
            - template indexes like `axion/templates/_INDEX.md`
            - any generated manifests
    - If integrity check fails → FAIL even if rebuild command returned 0
4. **Diff Capture (if requested_outputs.diff=true)**
    - Capture git diff (or filesystem diff) into:
        - `artifacts/diff.patch` or `artifacts/diff.md`
5. **Write Report**
    - PASS only if:
        - rebuild command succeeded
        - integrity checks passed

Proof Requirements (Locked)

On PASS, `report.proof[]` must include:

- `{ kind: "registry_build_pass", value: "rebuild", evidence_path: "runlog.md" }`

If diff requested, include:

- `{ kind: "diff_generated", value: "rebuild-diff", evidence_path: "artifacts/diff.patch" }` *(or diff.md)*

Required `report.results[]` entries (minimum)

- `check_id: "rebuild_command"` (PASS/FAIL)
- `check_id: "integrity_check"` (PASS/FAIL)
- `check_id: "diff_capture"` (PASS/FAIL/WARN, depending on requested_outputs.diff)

Each with pointers on failure.

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Optional (recommended for MM-12):

- `artifacts/diff.patch` (if requested_outputs.diff=true)

Failure Semantics

- If rebuild command fails → FAIL
- If rebuild command passes but integrity check fails → FAIL
- If diff capture fails (requested) → WARN in v1, FAIL in strict policy later

Mode Registry Entry (switch stub to real)

Update `maintenance/registry/modes.json` runner:

- from `runners/mm-stub-not-implemented.js`

- to `runners/mm-12-rebuild-indexes.js`

Also update `default_constraints` for MM-12 (recommended):

- `allow_writes: true`
- `outputs.diff: true`

Runner File Name

`maintenance/runners/mm-12-rebuild-indexes.js`

Runbook File Name

`maintenance/runbooks/MM-12.rebuild-indexes.md`

Example runspec (MM-12)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-12",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Rebuild indexes/registries" }
}
```

MM-20 Failure Triage Mode

P1.1.4 — MM-20 Failure Triage Mode (Mode Spec)

Mode ID

MM-20

Name

Failure Triage Mode

Purpose

When something breaks, MM-20 captures a **reproducible failure record** and a **structured evidence bundle** without making changes to the repo.

This mode is about:

- reproduce (if possible)
- isolate (scope hypothesis)
- record (commands, outputs, pointers, last known good)

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`

Required Capabilities

- `node` *(recommended for runner tooling)*
- `git` *(recommended for commit/branch history)*
- `npm` *(only if reproduction uses npm scripts)*

Runspec Inputs Used

- `target.target_id`
- `scope` (informational; used to narrow evidence collection if provided)
- `constraints.allow_writes` (**must be false**)
- `metadata.notes`
- Additional triage inputs should be provided as **notes** or via a companion artifact (see below).

Required Companion Artifact (triage write-up)

MM-20 must produce:

- `maintenance/runs/<run_id>/artifacts/triage.md`

This artifact is the “human-readable” triage record.

Steps (Deterministic)

1. **Preflight**
    - Confirm target exists
    - Confirm `allow_writes=false`
    - Record environment info:
        - node version
        - npm version (if available)
        - OS/platform (optional)
    - Record git metadata (if git available):
        - branch
        - commit hash
        - dirty/clean status
2. **Capture Inputs**
    - Read triage instructions from:
        - `runspec.metadata.notes` (minimum)
        - optionally: `artifacts/triage_inputs.json` if you later add this (v2)
3. **Attempt Reproduction (best-effort)**
    - If a failing command is provided (recommended), run it.
    - If not provided, do not guess “the failure”; just document missing repro command as WARN.
4. **Evidence Collection**
    - Capture:
        - command + exit code
        - trimmed stdout/stderr
        - file list of key folders (optional)
        - recent git diff summary (read-only)
5. **Write `triage.md`**
   Include required headers (below) and include:
    - exact reproduction command
    - observed outputs
    - suspected scope
    - immediate next action suggestions limited to *triage* (not solutions unless requested)
6. **Write Report**
    - PASS means: triage artifacts successfully created (even if repro fails)
    - WARN means: cannot reproduce (no failing command provided or non-deterministic failure)

    - FAIL means: runner couldn't create the triage artifacts / couldn't read target

Proof Requirements (Locked)

On PASS/WARN, `report.proof[]` must include:

- `{ kind: "triage_report_generated", value: "triage", evidence_path: "artifacts/triage.md" }`

Required `report.results[]` entries (minimum)

- `check_id: "triage_artifact"` (PASS/FAIL)
- `check_id: "repro_attempt"` (PASS/FAIL/WARN)
- `check_id: "evidence_capture"` (PASS/FAIL)

`triage.md` Required Headers (Locked)

- `# Triage`
- `## Symptom`
- `## Reproduction Steps`
- `## Failing Commands`
- `## Last Known Good`
- `## Suspected Scope`
- `## Evidence Collected`
- `## Next Actions`

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/triage.md`

Optional:

- `artifacts/repro_output.txt` (raw output capture if you want)

Failure Semantics

- MM-20 should almost never FAIL.

- It is acceptable to return WARN when reproduction is incomplete.

Mode Registry Entry (switch stub to real)

Update `maintenance/registry/modes.json` runner:

- from `runners/mm-stub-not-implemented.js`
- to `runners/mm-20-triage.js`

Update recommended constraints:

- `allow_writes: false`

Runner File Name

`maintenance/runners/mm-20-triage.js`

Runbook File Name

`maintenance/runbooks/MM-20.triage.md`

Example runspec (MM-20)


```
{
  "run_id": "placeholder",
  "mode_id": "MM-20",
  "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
  "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
  "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
  "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
  "expected_proof": [{ "kind": "required_from_modes_json", "value":
"required_from_modes_json" }],
  "metadata": {
    "env": "local",
    "requested_by": "agent",
    "notes": "Symptom: verify fails. Repro: npm test. Last good: <commit>."
  }
}
```

MM-21 Emergency Recovery Mode

P1.1.5 — MM-21 Emergency Recovery Mode (Mode Spec)

Mode ID

MM-21

Name

Emergency Recovery Mode

Purpose

Restore the target workspace to a **stable** or at least **verifiable** state after breakage. MM-21 is allowed to write changes, but only under explicitly declared recovery steps.

This mode is designed for situations like:

- verification suite is broken
- registries/indexes are corrupted
- partial upgrades left repo inconsistent
- you need to roll back to a known good snapshot/commit

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`

Required Capabilities

- `node`
- `git` *(strongly recommended; required if recovery uses rollback/restore by commit/tag)*
- `npm` *(optional; required if "post-recovery verify" is* `npm test` *etc.)*

Runspec Inputs Used

- `target.target_id`
- `scope` *(informational; recovery is usually whole target)*
- `constraints.allow_writes` (**must be true**)
- `constraints.allow_network` (default false)
- `requested_outputs.snapshot` (often true)
- `metadata.notes` (must include recovery objective + allowed actions)

Required Companion Artifact (recovery plan)

MM-21 must produce:

- `maintenance/runs/<run_id>/artifacts/recovery_plan.md`

This is required because recovery can be destructive.

Recovery Action Classes (Locked v1)

MM-21 supports these action classes. The runner should implement them in this order of preference (least destructive first):

1. **Registry/Index Rebuild**
    - Invoke MM-12-like rebuild procedure (directly or by calling rebuild command)
2. **Artifact/Config Repair (Scoped)**
    - Apply narrowly scoped repairs (only if explicitly listed in plan)
3. **Restore from Snapshot**
    - Unpack a known snapshot bundle to restore files (requires snapshot reference)
4. **Git Revert/Rollback**
    - Checkout/reset to known good commit/tag (requires explicit approval in plan)

If an action class is not applicable (missing inputs/capabilities), the runner records WARN/FAIL for that check and proceeds to the next allowed action *only if the plan allows it*.

Steps (Deterministic)

1. **Preflight**
    - Confirm target exists
    - Confirm `allow_writes=true`
    - Record git metadata (branch/commit, dirty status)
    - Parse recovery objective + allowed actions from `metadata.notes` (v1) and/or plan content (v2)
2. **Write Recovery Plan Artifact**
    - Generate `artifacts/recovery_plan.md` with:
        - objective
        - inputs (snapshot/run_id/commit)
        - allowed actions
        - step list
        - post-recovery verify command
3. **Execute Recovery Actions**
   For each enabled action class, attempt execution and record:
    - command(s) run
    - exit codes
    - what files changed (best-effort via git diff)
    - pointers on failure
4. **Post-Recovery Verification**

    - Always attempt a verify command at the end (if available):
        - use `targets.json.runtime_contract.verify`
    - Record pass/fail
5. **Write Report**
    - PASS if:
        - at least one recovery action succeeded **and**
        - post-recovery verify passes (preferred)
    - WARN if:
        - recovery actions succeeded but verify still fails
    - FAIL if:
        - no recovery action could be applied or target inaccessible

Proof Requirements (Locked)

On PASS/WARN, `report.proof[]` must include:

- `{ kind: "recovery_actions_applied", value: "recovery", evidence_path: "artifacts/recovery_plan.md" }`

If verify passes:

- `{ kind: "verify_pass", value: "post-recovery", evidence_path: "runlog.md" }`

Required `report.results[]` entries (minimum)

- `check_id: "recovery_plan"` (PASS/FAIL)
- `check_id: "action_rebuild_indexes"` (PASS/FAIL/WARN)

- `check_id: "action_restore_snapshot"` (PASS/FAIL/WARN)
- `check_id: "action_git_rollback"` (PASS/FAIL/WARN)
- `check_id: "post_recovery_verify"` (PASS/FAIL/WARN)

(Include only the actions your v1 runner supports; unsupported actions should be WARN with pointers describing “not implemented yet”.)

`recovery_plan.md` Required Headers (Locked)

- `# Recovery Plan`
- `## Objective`
- `## Inputs (snapshot/run_id/commit)`
- `## Allowed Actions`
- `## Steps`
- `## Post-Recovery Verification`

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/recovery_plan.md`

Optional (recommended):

- `artifacts/diff.patch` (what changed during recovery)
- `artifacts/snapshot.zip` (if requested_outputs.snapshot=true)

Failure Semantics

- MM-21 should prefer WARN over FAIL when partial recovery is possible.
- The mode must never perform destructive operations unless explicitly listed as allowed.

Mode Registry Entry (switch stub to real)

Update `maintenance/registry/modes.json` runner:

- from `runners/mm-stub-not-implemented.js`
- to `runners/mm-21-recovery.js`

Update recommended constraints:

- `allow_writes: true`
- `outputs.snapshot: true` *(often)*

Runner File Name

`maintenance/runners/mm-21-recovery.js`

Runbook File Name

`maintenance/runbooks/MM-21.recovery.md`

Example runspec (MM-21)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-21",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
```

"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": false },
"requested_outputs": { "diff": true, "snapshot": true, "changelog": false },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": {
"env": "local",
"requested_by": "agent",
"notes": "Objective: restore verify pass. Allowed actions: rebuild_indexes, restore_snapshot. Input snapshot: <path or run_id>."
}
}

Change & Release

MM-05 Planned Upgrade

P2.1.1 — MM-05 Planned Upgrade (Mode Spec)

Mode ID

MM-05

Name

Planned Upgrade

Purpose

Apply a planned change to the Axion system (templates/config/engine/UI/etc.) using a **dependency-closure-based upgrade plan**, then verify and record proof.

This is your standard “safe upgrade” mode.

Applies To

`allowed_target_types`: `axion_core | ui | templates`

Required Capabilities

- `node`
- `git` *(strongly recommended; required for diff/changelog discipline)*
- `npm` *(if verify/build commands rely on npm scripts)*

Runspec Inputs Used

- `target.target_id`
- `scope` (**must be explicit**; `paths` or `components` recommended)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (recommended true)

- `requested_outputs.changelog` (recommended true)
- `metadata.notes` (must reference what is being upgraded)

Required Companion Artifact (Upgrade Plan)

MM-05 must produce (or require as input) an upgrade plan:

- `maintenance/runs/<run_id>/artifacts/upgrade_plan.md`

Locked rule: No MM-05 run is valid without this artifact.

Upgrade Plan Content Requirements (Locked)

`upgrade_plan.md` must include:

- `## Summary`
- `## Scope`
- `## Dependency Closure`
    - `### Direct Dependencies`
    - `### Reverse Dependencies`
    - `### Transitive Dependencies`
    - `### Companion Upgrades (Patch Set)`
- `## Compatibility`
- `## Migration Plan` *(only if needed)*
- `## Verification Proof Required`

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm scope is not empty
    - Confirm upgrade plan artifact exists (or generate it at run start if your workflow does that)
    - Record git metadata + “before” diff state (if git available)
2. **Apply Patch Set**
    - Apply changes only within scope boundaries
    - If changes exceed scope → FAIL (or WARN + fail in strict policy)
3. **Verification**
    - Run verify command from `targets.json.runtime_contract.verify`
    - Optionally run build command from `runtime_contract.build` if upgrade affects runtime
4. **Diff Capture (if requested_outputs.diff=true)**
    - Produce `artifacts/diff.patch` (preferred) or `diff.md`
5. **Changelog Update (if requested_outputs.changelog=true)**
    - Append a changelog entry:
        - either in repo changelog (if you manage one)
        - or in `artifacts/changelog.md` (maintenance-owned)
6. **Write Report**
    - PASS only if verification passes and required artifacts exist

Proof Requirements (Locked)

On PASS, `report.proof[]` must include:

- `{ kind: "verify_pass", value: "post-upgrade", evidence_path: "runlog.md" }`

If diff requested:

- `{ kind: "diff_generated", value: "upgrade-diff", evidence_path: "artifacts/diff.patch" }`

If changelog requested:

- `{ kind: "changelog_written", value: "upgrade-changelog", evidence_path: "artifacts/changelog.md" }` *(or repo changelog path)*

Required `report.results[]` entries (minimum)

- `upgrade_plan_present` (PASS/FAIL)
- `scope_enforced` (PASS/FAIL)
- `patch_applied` (PASS/FAIL)
- `post_upgrade_verify` (PASS/FAIL)
- `diff_capture` (PASS/FAIL/WARN)
- `changelog_update` (PASS/FAIL/WARN)

Outputs

Always:

- `runspec.json`
- `runlog.md`

- `report.json`
- `artifacts/upgrade_plan.md`

Optional (recommended):

- `artifacts/diff.patch`
- `artifacts/changelog.md`

Failure Semantics

- Missing upgrade plan → FAIL
- Verification fails → FAIL
- Scope violated → FAIL

Mode Registry Entry (switch stub to real)

Update `maintenance/registry/modes.json` runner:

- to `runners/mm-05-planned-upgrade.js`

Runner File Name

`maintenance/runners/mm-05-planned-upgrade.js`

Runbook File Name

`maintenance/runbooks/MM-05.planned-upgrade.md`

Example runspec (MM-05)

```
{
 "run_id": "placeholder",
 "mode_id": "MM-05",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
```

"scope": { "type": "paths", "paths": ["axion/templates/", "axion/registry/"], "components": [], "diff": { "from": "", "to": "" } },
"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": false },
"requested_outputs": { "diff": true, "snapshot": false, "changelog": true },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": { "env": "local", "requested_by": "agent", "notes": "Upgrade: template system inventory + registry sync" }
}

MM-06 Hotfix

P2.1.2 — MM-06 Hotfix (Mode Spec)

Mode ID

MM-06

Name

Hotfix

Purpose

Apply a **minimal, urgent fix** with stricter boundaries than MM-05. Hotfixes are:

- narrow scope
- minimal changes
- maximum proof
- no refactors unless explicitly authorized

Applies To

`allowed_target_types`: `axion_core | ui | templates`

Required Capabilities

- `node`
- `git` *(strongly recommended; required for diff discipline)*
- `npm` *(if verify uses npm)*

Runspec Inputs Used

- `target.target_id`
- `scope` (**must be explicit and narrow**; `paths` required in v1)

- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**must be true**)
- `requested_outputs.changelog` (**must be true**)
- `metadata.notes` (must include incident/fix summary)

Hotfix Hard Constraints (Locked)

MM-06 enforces the following rules:

1. **Scope must be paths-based**
    - `scope.type = "paths"`
    - at least one explicit path
2. **No renames / no moves by default**
    - if rename/move is needed, it must be declared in the hotfix plan
3. **Verification required**
    - post-fix verify must run and pass
4. **Diff + changelog required**
    - must produce diff artifact
    - must produce changelog entry

Required Companion Artifact (Hotfix Plan)

MM-06 must produce:

- `maintenance/runs/<run_id>/artifacts/hotfix_plan.md`

Locked rule: No MM-06 run is valid without this artifact.

Hotfix Plan Content Requirements (Locked)

`hotfix_plan.md` must include:

- `## Incident Summary`
- `## Fix Summary`
- `## Scope (Paths)`
- `## Risk Assessment`
- `## Verification Proof Required`
- `## Rollback Plan`

Steps (Deterministic)

1. **Preflight**
    - Confirm allow_writes=true
    - Confirm scope is paths-based and narrow
    - Confirm hotfix plan exists
    - Capture git metadata and pre-change status
2. **Apply Minimal Patch**
    - Apply changes strictly within declared paths
    - Reject changes outside scope (FAIL)
3. **Verification**
    - Run `targets.json.runtime_contract.verify`
    - (Optional) run build if incident suggests runtime breakage

4. **Diff Capture**
    - Write `artifacts/diff.patch`
5. **Changelog Update**
    - Write `artifacts/changelog.md` (or update repo changelog)
6. **Write Report**
    - PASS only if verify passes and artifacts exist

Proof Requirements (Locked)

On PASS:

- `{ kind: "verify_pass", value: "post-hotfix", evidence_path: "runlog.md" }`
- `{ kind: "diff_generated", value: "hotfix-diff", evidence_path: "artifacts/diff.patch" }`
- `{ kind: "changelog_written", value: "hotfix-changelog", evidence_path: "artifacts/changelog.md" }`

Required `report.results[]` entries (minimum)

- `hotfix_plan_present` (PASS/FAIL)
- `scope_enforced` (PASS/FAIL)
- `patch_applied` (PASS/FAIL)
- `post_hotfix_verify` (PASS/FAIL)
- `diff_capture` (PASS/FAIL)
- `changelog_update` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/hotfix_plan.md`
- `artifacts/diff.patch`
- `artifacts/changelog.md`

Failure Semantics

- Any scope violation → FAIL
- Verify fail → FAIL
- Missing diff/changelog → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-06-hotfix.js`

Also update mode defaults (recommended):

- `outputs.diff = true`
- `outputs.changelog = true`

Runner File Name

`maintenance/runners/mm-06-hotfix.js`

Runbook File Name

`maintenance/runbooks/MM-06.hotfix.md`

Example runspec (MM-06)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-06",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/scripts/", "axion/config/"], "components": [], "diff": {
"from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": false, "changelog": true },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Hotfix: restore verify passing
after config regression" }
}
```

MM-16 Snapshot / Archive

P2.2.1 — MM-16 Snapshot / Archive (Mode Spec)

Mode ID

MM-16

Name

Snapshot / Archive

Purpose

Create a **frozen, portable snapshot** of the target workspace (or selected scope) so you can:

- audit past states
- diff against baseline
- recover from breakage
- package a release baseline

This mode is primarily read-only relative to the repo (it writes snapshot artifacts under the run folder).

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`

Required Capabilities

- `node` *(or shell zip tooling if runner is shell-based)*
- `git` *(optional but recommended for commit metadata)*

Runspec Inputs Used

- `target.target_id`

- `scope` (all/paths/components supported; v1 recommended: `paths` or `all`)
- `constraints.allow_writes` (should be `false`; only writes under maintenance run dir)
- `requested_outputs.snapshot` (**must be true**)
- `metadata.notes`

Snapshot Contents (Locked v1)

A snapshot must include, at minimum:

- the target files within the declared scope
- a snapshot manifest:
    - `artifacts/snapshot_manifest.json`
- the mode run artifacts:
    - `runspec.json`
    - `runlog.md`
    - `report.json`

Snapshot Format (Locked v1)

- `artifacts/snapshot.zip`

(You can add `.tar.gz` later, but v1 locks zip.)

Steps (Deterministic)

1. **Preflight**
    - Confirm `requested_outputs.snapshot=true`

    - Confirm scope is valid
    - Record git metadata (branch/commit) if available

2. **Collect File Set**
    - If scope.type = `all`: include entire target root **excluding**:
        - `.git/`
        - `maintenance/runs/`
        - `node_modules/` *(optional exclusion; recommended)*
    - If scope.type = `paths`: include only those paths
    - If scope.type = `components`: resolve to paths using a mapping (v2). In v1, components is WARN unless mapping exists.
3. **Write Snapshot Manifest**
    - `artifacts/snapshot_manifest.json` includes:
        - run_id
        - target_id
        - included paths
        - excluded paths
        - commit/branch (if available)
        - timestamp
4. **Create Zip**
    - Write: `artifacts/snapshot.zip`
    - Ensure manifest is inside the zip.
5. **Write Report**

    - PASS only if snapshot.zip exists and manifest exists.

Proof Requirements (Locked)

On PASS:

- `{ kind: "snapshot_exists", value: "snapshot", evidence_path: "artifacts/snapshot.zip" }`

Required `report.results[]` entries (minimum)

- `file_set_resolved` (PASS/FAIL/WARN)
- `manifest_written` (PASS/FAIL)
- `snapshot_zip_created` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/snapshot_manifest.json`
- `artifacts/snapshot.zip`

Failure Semantics

- Missing zip → FAIL

- Missing manifest → FAIL
- Unsupported scope mapping (components without mapping) → WARN in v1 (or FAIL in strict policy later)

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-16-snapshot.js`

Update mode defaults (recommended):

- `outputs.snapshot = true`
- `default_constraints.allow_writes = false`

Runner File Name

`maintenance/runners/mm-16-snapshot.js`

Runbook File Name

`maintenance/runbooks/MM-16.snapshot.md`

Example runspec (MM-16)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-16",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/", "client/", "server/"], "components": [], "diff": { "from":
"", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": true, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Snapshot before upgrade" }
}
```

MM-14 Release Packaging

P2.2.2 — MM-14 Release Packaging (Mode Spec)

Mode ID

MM-14

Name

Release Packaging

Purpose

Produce a **release bundle** of the target system with:

- version metadata
- changelog entry (or changelog artifact)
- packaged archive
- checksums/manifest
- post-package verification proof

This is the “make something shippable” mode.

Applies To

`allowed_target_types`: `axion_core` *(locked v1)*

Required Capabilities

- `node`
- `npm` *(if build/verify uses npm)*
- `git` *(required in v1 for release metadata + rollback support)*

Runspec Inputs Used

- `target.target_id`
- `scope` (v1: typically `all`)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.snapshot` (**recommended true**; release bundle is effectively a snapshot)
- `requested_outputs.changelog` (**must be true**)
- `metadata.notes` (must include version intent)

Release Inputs (Locked v1)

MM-14 requires the release to specify one of:

- `version` (explicit semver) OR
- `version_bump` (major|minor|patch)

In v1, since runspec schema doesn't include these fields, they must be carried in:

- `metadata.notes` **and/or**
- `artifacts/release_plan.md` (recommended)

Required Companion Artifact (Release Plan)

MM-14 must produce:

- `maintenance/runs/<run_id>/artifacts/release_plan.md`

Required headers:

- `# Release Plan`
- `## Version`

- `## Summary`
- `## Build Steps`
- `## Verification Steps`
- `## Packaging Outputs`
- `## Rollback Reference`

Release Outputs (Locked v1)

Stored under `maintenance/runs/<run_id>/artifacts/`:

- `release.zip` *(the packaged output)*
- `release_manifest.json` *(metadata + file list)*
- `checksums.txt` *(sha256 list)*
- `changelog.md` *(or repo changelog updated — but v1 recommends maintenance-owned changelog artifact)*

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm git is available and repo is in a known state (dirty allowed or not — policy decision; v1 should WARN if dirty)
    - Write `release_plan.md` from notes/version inputs
2. **Versioning**
    - Apply version bump to the chosen version source:
        - Option A: repo `package.json` version (if this is your canonical version)

        - Option B: maintenance-only version recorded in manifest (no repo change)

    - v1 recommendation for safety: **record version in manifest + changelog artifact** and avoid repo-wide version mutation unless you explicitly want it.

3. **Build**

    - Run `targets.json.runtime_contract.build` (if defined)

    - Capture output

4. **Verify**

    - Run `targets.json.runtime_contract.verify`

    - Must pass for a PASS release

5. **Package**

    - Produce `artifacts/release.zip`

    - Include:

        - relevant target files (usually whole repo minus exclusions)

        - run artifacts (runspec/runlog/report/manifest/checksums/changelog)

    - Exclude:

        - `.git/`

        - `maintenance/runs/` *(except the current run folder artifacts if you choose to embed them)*

        - `node_modules/` *(recommended)*

6. **Checksums + Manifest**

    - Write `checksums.txt` (sha256 per file inside release zip or for the zip itself; choose one and lock later)

    - Write `release_manifest.json` containing:
        - version
        - timestamp
        - commit hash
        - build/verify commands used
        - artifact file list

7. **Write Report**
    - PASS only if:
        - verify passes
        - release.zip exists
        - release_manifest.json exists
        - checksums.txt exists
        - changelog exists/updated

Proof Requirements (Locked)

On PASS:

- `{ kind: "package_exists", value: "release", evidence_path: "artifacts/release.zip" }`
- `{ kind: "verify_pass", value: "release-verify", evidence_path: "runlog.md" }`

Recommended additional proof:

- `{ kind: "checksum_exists", value: "release-checksums", evidence_path: "artifacts/checksums.txt" }`

- `{ kind: "manifest_exists", value: "release-manifest", evidence_path: "artifacts/release_manifest.json" }`

Required `report.results[]` entries (minimum)

- `release_plan` (PASS/FAIL)
- `build` (PASS/FAIL/WARN if build not defined)
- `verify` (PASS/FAIL)
- `package` (PASS/FAIL)
- `manifest` (PASS/FAIL)
- `checksums` (PASS/FAIL)
- `changelog` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/release_plan.md`
- `artifacts/release.zip`
- `artifacts/release_manifest.json`

- `artifacts/checksums.txt`
- `artifacts/changelog.md` *(or repo changelog updated, but v1 prefers artifact)*

Failure Semantics

- Verify fails → FAIL (no release PASS)
- Packaging missing → FAIL
- Missing manifest/checksums/changelog → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-14-release-package.js`

Update mode defaults (recommended):

- `outputs.snapshot = true`
- `outputs.changelog = true`

Runner File Name

`maintenance/runners/mm-14-release-package.js`

Runbook File Name

`maintenance/runbooks/MM-14.release-package.md`

Example runspec (MM-14)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-14",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
```

"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": false },
"requested_outputs": { "diff": false, "snapshot": true, "changelog": true },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": {
"env": "prod",
"requested_by": "agent",
"notes": "Release: version=1.2.0; summary=Template system v2 rollout"
}
}

MM-15 Rollback / Revert Release

P2.2.3 — MM-15 Rollback / Revert Release (Mode Spec)

Mode ID

MM-15

Name

Rollback / Revert Release

Purpose

Restore the target workspace to a **previous known good state** after a bad release/upgrade. This is the controlled “go back” mode.

Rollback can be performed using:

- a snapshot archive created by MM-16/MM-14
- a git commit/tag reference (preferred if available)

Applies To

`allowed_target_types`: `axion_core` *(locked v1)*

Required Capabilities

- `git` *(required in v1 if using commit/tag rollback)*
- `node` *(for runner + report writing)*
- `npm` *(if post-rollback verify uses npm scripts)*

Runspec Inputs Used

- `target.target_id`
- `constraints.allow_writes` (**must be true**)

- `requested_outputs.snapshot` (optional; can snapshot before rollback)
- `metadata.notes` (must include rollback reference)

Rollback Inputs (Locked v1)

Rollback requires one of these references (must be specified in notes and/or artifact):

- `rollback_commit_or_tag: <ref>`
  OR
- `rollback_snapshot_path: <path to snapshot.zip>`
  OR
- `rollback_run_id: <run_id>` (runner resolves snapshot from that run folder)

In v1, because runspec schema doesn't include dedicated fields, these are carried in:

- `metadata.notes`
- and/or `artifacts/rollback_plan.md` (recommended)

Required Companion Artifact (Rollback Plan)

MM-15 must produce:

- `maintenance/runs/<run_id>/artifacts/rollback_plan.md`

Required headers:

- `# Rollback Plan`
- `## Rollback Target (commit/tag or snapshot)`
- `## Pre-Rollback Snapshot`
- `## Steps`

- `## Post-Rollback Verification`
- `## Outcome Criteria`

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm rollback reference exists (commit/tag or snapshot)
    - Record current git metadata and dirty status
2. **Optional Pre-Rollback Snapshot**
    - If `requested_outputs.snapshot=true`, run an internal “quick snapshot”:
        - create `artifacts/pre_rollback_snapshot.zip` (optional v1)
    - This is safety insurance.
3. **Execute Rollback**
   Path A — Git rollback
    - Use one of:
        - `git checkout <ref>` (non-destructive)
        - `git reset --hard <ref>` (destructive; requires explicit permission in rollback plan)
    - v1 recommendation: default to non-destructive checkout; only hard reset if explicitly allowed.
4. **Path B — Snapshot restore**
    - Unzip snapshot to target root according to rules:
        - do not restore `.git/`

        - do not overwrite `maintenance/`
    - Snapshot restore must be explicitly declared in plan.
5. **Post-Rollback Verification**
    - Run `targets.json.runtime_contract.verify`
    - Optionally run build if appropriate
6. **Diff Capture (optional)**
    - Write `artifacts/diff.patch` (what rollback changed) if requested.
7. **Write Report**
    - PASS only if post-rollback verify passes
    - WARN if rollback performed but verify still fails
    - FAIL if rollback cannot be executed or repo becomes inaccessible

Proof Requirements (Locked)

On PASS:

- `{ kind: "rollback_complete", value: "rollback", evidence_path: "artifacts/rollback_plan.md" }`
- `{ kind: "verify_pass", value: "post-rollback", evidence_path: "runlog.md" }`

Required `report.results[]` entries (minimum)

- `rollback_plan` (PASS/FAIL)
- `rollback_execute` (PASS/FAIL)
- `post_rollback_verify` (PASS/FAIL/WARN)

- `pre_rollback_snapshot` (PASS/FAIL/WARN if requested)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/rollback_plan.md`

Optional:

- `artifacts/pre_rollback_snapshot.zip`
- `artifacts/diff.patch`

Failure Semantics

- Missing rollback reference → FAIL
- Rollback executed but verify fails → WARN (or FAIL in strict policy)
- Any destructive action without explicit permission → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-15-rollback.js`

Update defaults (recommended):

- `outputs.snapshot = true` *(so you can snapshot before rollback)*

Runner File Name

`maintenance/runners/mm-15-rollback.js`

Runbook File Name

`maintenance/runbooks/MM-15.rollback.md`

Example runspec (MM-15)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-15",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": true, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": {
   "env": "prod",
   "requested_by": "agent",
   "notes": "Rollback: rollback_commit_or_tag=v1.1.3; destructive_allowed=no"
 }
}
```

Full Coverage

MM-03 Coverage Audit

P3.1.1 — MM-03 Coverage Audit (Mode Spec)

Mode ID

MM-03

Name

Coverage Audit

Purpose

Compute and report **coverage deltas** across Axion's required artifacts/templates/packs (what exists vs what is required), without making changes.

Outputs a structured "what's missing / what's stale" report that can drive upgrades.

Applies To

`allowed_target_types`: `axion_core | templates`

Required Capabilities

- `node`
- `git` *(optional; for metadata)*

Runspec Inputs Used

- `target.target_id`
- `scope` (v1: informational; v2 can scope coverage to domains/components)
- `constraints.allow_writes` (**must be false**)
- `requested_outputs.diff` (optional; if you want to diff coverage against baseline later)
- `metadata.notes`

Coverage Source-of-Truth (Locked v1)

MM-03 reads coverage requirements from one of these sources (in this priority order):

1. **Axion config coverage map** (preferred):
   e.g., `axion/config/coverage_map.json` (or equivalent)

2. **Maintenance-defined coverage config** (fallback):
   e.g., `maintenance/registry/coverage_rules.json` *(optional future file; not required now)*

If no coverage rules are found, mode returns **FAIL** with pointers to the missing config.

Steps (Deterministic)

1. **Preflight**
   - Confirm target exists
   - Confirm `allow_writes=false`
   - Locate coverage rules source
2. **Inventory**
   - Scan the target filesystem for artifacts/templates/packs that count toward coverage
   - Build an inventory list:
     - counts by category/domain
     - missing required items
     - unexpected extras (optional)
3. **Compute Coverage**
   - Compute:
     - total required

        - total present
        - percent coverage
        - missing list (with paths/IDs)
        - stale/invalid items (optional, if detectable)

4. **Write Coverage Report Artifact**
    - `maintenance/runs/<run_id>/artifacts/coverage_report.json`
    - Optionally also write a human summary:
        - `artifacts/coverage_summary.md`

5. **Write Report**
    - PASS if coverage report produced successfully
    - WARN if coverage is below thresholds (policy choice; default WARN)
    - FAIL only for operational failure (missing rules, scan failure, etc.)

Proof Requirements (Locked)

On PASS/WARN:

- `{ kind: "coverage_report_generated", value: "coverage", evidence_path: "artifacts/coverage_report.json" }`

Required `report.results[]` entries (minimum)

- `coverage_rules_loaded` (PASS/FAIL)
- `inventory_built` (PASS/FAIL)
- `coverage_computed` (PASS/FAIL)
- `coverage_report_written` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/coverage_report.json`

Optional:

- `artifacts/coverage_summary.md`

Failure Semantics

- Missing coverage rules source → FAIL
- Inventory/scan failure → FAIL
- Low coverage → WARN (unless strict policy later)

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-03-coverage-audit.js`

Runner File Name

`maintenance/runners/mm-03-coverage-audit.js`

Runbook File Name

`maintenance/runbooks/MM-03.coverage-audit.md`

Example runspec (MM-03)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-03",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Coverage audit for axion-core" }
}
```

MM-04 Drift Detection

P3.1.2 — MM-04 Drift Detection (Mode Spec)

Mode ID

MM-04

Name

Drift Detection

Purpose

Detect unintended drift between:

- current repo state and a baseline (commit/tag), **or**
- two snapshots/run IDs, **or**
- current state and expected generated outputs

This mode produces a **diff artifact** and a **drift report** so upgrades can be evaluated objectively.

Applies To

`allowed_target_types`: `axion_core | ui | templates | other`

Required Capabilities

- `git` *(recommended; required for commit-based drift)*
- `node` *(for runner/report writing)*
- Optional: `diff` tooling (runner can use git diff)

Runspec Inputs Used

- `target.target_id`
- `scope` (paths/diff_range supported; v1 recommended: `paths` or `diff_range`)

- `constraints.allow_writes` (**must be false**)
- `requested_outputs.diff` (**must be true**)
- `metadata.notes` (must include baseline reference)

Drift Baseline Inputs (Locked v1)

MM-04 requires one baseline method declared in notes and/or artifact:

A) Git baseline

- `baseline_commit_or_tag: <ref>`
- and optionally `compare_to: HEAD`

B) Snapshot baseline

- `baseline_snapshot_path: <snapshot.zip>`
- or `baseline_run_id: <run_id>`

Because runspec schema doesn't include these fields, they are carried in:

- `metadata.notes`
- and/or `artifacts/drift_plan.md` (recommended)

Required Companion Artifact (Drift Plan)

MM-04 must produce:

- `maintenance/runs/<run_id>/artifacts/drift_plan.md`

Required headers:

- `# Drift Plan`

- `## Baseline Type (git|snapshot)`
- `## Baseline Reference`
- `## Scope`
- `## Diff Output`
- `## Interpretation Notes`

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=false`
    - Confirm `requested_outputs.diff=true`
    - Confirm baseline method is provided
    - Write drift_plan.md
2. **Resolve Comparison**
    - If git baseline:
        - run `git diff <baseline>..HEAD -- <paths>` (paths optional)
    - If snapshot baseline:
        - extract snapshot to temp dir (outside repo)
        - compute file diffs between extracted baseline and current target paths
3. **Write Diff Artifact**
    - `artifacts/diff.patch` (preferred)
    - If patch not possible (snapshot compare), write:

        - `artifacts/diff.md` with structured file-by-file changes

4. **Write Drift Report**
    - `artifacts/drift_report.json` containing:
        - baseline reference
        - changed file list
        - counts (added/modified/deleted)
        - summary text
5. **Write Report**
    - PASS if diff + drift report are produced successfully
    - WARN if drift is large (policy choice; default WARN only if thresholds are configured)
    - FAIL only on operational failure (missing baseline, diff failure, etc.)

Proof Requirements (Locked)

On PASS/WARN:

- `{ kind: "diff_generated", value: "drift", evidence_path: "artifacts/diff.patch" }` *(or diff.md)*

Recommended additional proof:

- `{ kind: "drift_report_generated", value: "drift-report", evidence_path: "artifacts/drift_report.json" }`

Required `report.results[]` entries (minimum)

- `baseline_resolved` (PASS/FAIL)

- `diff_generated` (PASS/FAIL)
- `drift_report_written` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/drift_plan.md`
- `artifacts/diff.patch` *(or diff.md)*
- `artifacts/drift_report.json`

Failure Semantics

- Missing baseline reference → FAIL
- Diff generation fails → FAIL
- Drift “too large” is not a failure in v1 unless you configure thresholds later.

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-04-drift-detect.js`

Update defaults (recommended):

- `outputs.diff = true`
- `default_constraints.allow_writes = false`

Runner File Name

`maintenance/runners/mm-04-drift-detect.js`

Runbook File Name

`maintenance/runbooks/MM-04.drift-detect.md`

Example runspec (MM-04)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-04",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "diff_range", "paths": ["axion/", "maintenance/registry/"], "components": [],
"diff": { "from": "v1.1.0", "to": "HEAD" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Baseline:
baseline_commit_or_tag=v1.1.0" }
}
```

MM-17 Dependency Refresh

P3.1.3 — MM-17 Dependency Refresh (Mode Spec)

Mode ID

MM-17

Name

Dependency Refresh

Purpose

Update runtime dependencies (and lockfiles) in a controlled way, then verify the system still passes. This mode exists to manage “dependency drift” intentionally rather than accidentally.

Applies To

`allowed_target_types`: `axion_core | ui`

Required Capabilities

- `node`
- `npm` *(or pnpm/yarn — pick one later; v1 assumes npm based on targets.json)*
- `git` *(required in v1 for diff/audit and rollback discipline)*
- Network access is typically required for installs.

Runspec Inputs Used

- `target.target_id`
- `scope` (v1: informational; dependencies usually touch whole repo)
- `constraints.allow_writes` (**must be true**)
- `constraints.allow_network` (**must be true**)

- `requested_outputs.diff` (**must be true**)
- `requested_outputs.changelog` (recommended true)
- `metadata.notes` (must include dependency intent)

Dependency Refresh Inputs (Locked v1)

MM-17 must have an explicit intent statement (in notes or a plan artifact):

- “update all minor/patch”
- “update specific packages”
- “regenerate lockfile”
- “security patch only”

Because runspec schema doesn’t include structured fields yet, the intent must be captured in:

- `metadata.notes`
- and/or `artifacts/deps_plan.md` (recommended)

Required Companion Artifact (Dependency Plan)

MM-17 must produce:

- `maintenance/runs/<run_id>/artifacts/deps_plan.md`

Required headers:

- `# Dependency Refresh Plan`
- `## Intent`
- `## Packages / Scope`

- `## Install Strategy`
- `## Verification`
- `## Rollback Plan`

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm `allow_network=true`
    - Confirm git available
    - Write `deps_plan.md`
2. **Dependency Update**
   Execute one of these strategies (locked v1: choose one and document it):
    - Strategy A (basic): `npm update`
    - Strategy B (controlled): update selected packages then `npm install`
    - Strategy C (lock reset): delete lockfile + `npm install` *(destructive; must be explicitly allowed in plan)*
3. **Lockfile & Manifest Check**
    - Ensure:
        - `package.json` still parses
        - lockfile exists and updated
    - If lockfile missing after install → FAIL
4. **Verification**

    - Run `targets.json.runtime_contract.verify`
    - Optionally run build
5. **Diff Capture**
    - Write `artifacts/diff.patch` capturing changes to `package.json` + lockfile (+ any transitive changes)
6. **Changelog**
    - If requested: write `artifacts/changelog.md` with dependency refresh entry
7. **Write Report**
    - PASS only if verify passes and diff exists

Proof Requirements (Locked)

On PASS:

- `{ kind: "deps_updated", value: "dependencies", evidence_path: "artifacts/diff.patch" }`
- `{ kind: "verify_pass", value: "post-deps-refresh", evidence_path: "runlog.md" }`

Recommended:

- `{ kind: "changelog_written", value: "deps-changelog", evidence_path: "artifacts/changelog.md" }`

Required `report.results[]` entries (minimum)

- `deps_plan` (PASS/FAIL)
- `deps_update` (PASS/FAIL)

- `lockfile_present` (PASS/FAIL)
- `post_update_verify` (PASS/FAIL)
- `diff_capture` (PASS/FAIL)
- `changelog_update` (PASS/FAIL/WARN)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/deps_plan.md`
- `artifacts/diff.patch`

Optional:

- `artifacts/changelog.md`

Failure Semantics

- `allow_network=false` → FAIL (dependencies can't refresh)
- install/update fails → FAIL
- verify fails → FAIL (no PASS refresh)
- missing lockfile → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-17-deps-refresh.js`

Update defaults (recommended):

- `default_constraints.allow_network = true`
- `outputs.diff = true`
- `outputs.changelog = true`

Runner File Name

`maintenance/runners/mm-17-deps-refresh.js`

Runbook File Name

`maintenance/runbooks/MM-17.deps-refresh.md`

Example runspec (MM-17)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-17",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": true
},
 "requested_outputs": { "diff": true, "snapshot": false, "changelog": true },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Intent: update minor/patch
versions; keep lockfile; verify must pass." }
}
```

MM-19 CI Sanity Run

P3.1.4 — MM-19 CI Sanity Run (Mode Spec)

Mode ID

MM-19

Name

CI Sanity Run

Purpose

Run the **CI-equivalent pipeline** locally (or in CI) in a deterministic, non-interactive way to ensure the repo is in a shippable/mergeable state.

Think: “simulate CI without actually pushing.”

Applies To

`allowed_target_types`: `axion_core | ui`

Required Capabilities

- `node`
- `npm`
- `git` *(recommended for metadata; optional otherwise)*

Runspec Inputs Used

- `target.target_id`
- `scope` (v1: informational; CI sanity is usually whole target)
- `constraints.allow_writes` (**must be false**)
- `constraints.allow_network` (default false; true only if CI sanity includes installs)

- `metadata.env` (ci recommended)
- `metadata.notes`

CI Pipeline Definition (Locked v1)

MM-19 must run a defined command set. Source-of-truth priority order:

1. `targets.json.runtime_contract.ci` *(if defined)*
2. fallback: `targets.json.runtime_contract.verify` *(minimum CI sanity)*

Recommendation: Add to targets.json later:

- `ci`: `"npm run ci"`

Steps (Deterministic)

1. **Preflight**
    - Confirm target exists
    - Confirm `allow_writes=false`
    - Resolve CI command(s)
    - Record environment info (node/npm versions)
2. **Execute CI Commands**
    - Run resolved CI command(s)
    - Capture exit codes and duration
3. **Write Report**
    - PASS if all CI commands exit 0
    - FAIL otherwise

    - Include pointers best-effort from output (optional v1)

Proof Requirements (Locked)

On PASS:

- `{ kind: "ci_pass", value: "ci", evidence_path: "runlog.md" }`

Required `report.results[]` entries (minimum)

- `ci_command_resolved` (PASS/FAIL)
- `ci_run` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Optional:

- none (read-only)

Failure Semantics

- CI command missing/unresolvable → FAIL
- Command non-zero exit → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-19-ci-sanity.js`

Update defaults (recommended):

- `default_constraints.allow_writes = false`
- `required_proof` should use `ci_pass`

Runner File Name

`maintenance/runners/mm-19-ci-sanity.js`

Runbook File Name

`maintenance/runbooks/MM-19.ci-sanity.md`

Example runspec (MM-19)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-19",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "all", "paths": [], "components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "ci", "requested_by": "agent", "notes": "CI sanity run (local equivalent)" }
}
```

MM-13 Registry Integrity Cleanup

P3.1.5 — MM-13 Registry Integrity Cleanup (Mode Spec)

Mode ID

MM-13

Name

Registry Integrity Cleanup

Purpose

Normalize and repair Axion registries/indexes to remove:

- orphan entries
- duplicates
- invalid IDs
- broken references
- stale entries not present in filesystem

This is a “cleanup and normalize” mode, typically used after lots of change or after recovering from corruption.

Applies To

`allowed_target_types`: `axion_core | templates`

Required Capabilities

- `node`
- `git` *(recommended for diff + audit trail)*

Runspec Inputs Used

- `target.target_id`
- `scope` (v1: informational; cleanup is usually specific paths)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**recommended true**)
- `metadata.notes` (must describe what registries are targeted)

Registry Targets (Locked v1)

The cleanup runner operates on a defined set of registry/index locations. Priority order:

1. If Axion defines registry locations (e.g., in config): use those.
2. Otherwise use maintenance-defined defaults (document them in the runbook), e.g.:
    - `axion/registry/`
    - `axion/templates/_INDEX.md`
    - any `*.index.json` or `manifest.json` locations you standardize

If no registry targets can be determined → FAIL with pointers.

Cleanup Operations (Locked v1 set)

For each registry target:

1. **Parse validation**
    - ensure file is readable and parseable (JSON/MD rules)
2. **Deduplication**
    - remove duplicate entries by ID
3. **Orphan removal**

    - remove entries that point to non-existent filesystem paths

4. **Normalization**
    - normalize ID formats (case, separators) per rules
5. **Reference integrity**
    - ensure internal references resolve (best-effort)
6. **Write back**
    - rewrite registry files in canonical formatting (stable ordering recommended)

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Resolve registry target set
    - Record pre-clean state (git status/diff)
2. **Execute Cleanup**
    - Perform operations above, producing:
        - a summary of what changed
        - a list of removed/modified entries
3. **Post-Cleanup Validation**
    - Re-parse all affected registry files
    - Optionally run a lightweight “registry integrity check” command if available
4. **Diff Capture (recommended)**
    - Write `artifacts/diff.patch`

5. **Write Report**
    - PASS if cleanup completed and post-validation passes

Proof Requirements (Locked)

On PASS:

- `{ kind: "registry_clean", value: "cleanup", evidence_path: "runlog.md" }`

If diff generated:

- `{ kind: "diff_generated", value: "registry-clean-diff", evidence_path: "artifacts/diff.patch" }`

Required `report.results[]` entries (minimum)

- `targets_resolved` (PASS/FAIL)
- `cleanup_applied` (PASS/FAIL)
- `post_validation` (PASS/FAIL)
- `diff_capture` (PASS/FAIL/WARN)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Optional (recommended):

- `artifacts/diff.patch`
- `artifacts/cleanup_summary.json` (structured change summary)

Failure Semantics

- If cleanup cannot parse registry → FAIL
- If post-validation fails → FAIL
- If registry targets missing → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-13-registry-cleanup.js`

Update defaults (recommended):

- `outputs.diff = true`
- `default_constraints.allow_writes = true`

Runner File Name

`maintenance/runners/mm-13-registry-cleanup.js`

Runbook File Name

`maintenance/runbooks/MM-13.registry-cleanup.md`

Example runspec (MM-13)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-13",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/registry/"], "components": [], "diff": { "from": "", "to": "" }
},
```

"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": false },
"requested_outputs": { "diff": true, "snapshot": false, "changelog": false },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": { "env": "local", "requested_by": "agent", "notes": "Cleanup registries after template reorg" }
}

MM-11 Data Repair / Salvage

P3.1.6 — MM-11 Data Repair / Salvage (Mode Spec)

Mode ID

MM-11

Name

Data Repair / Salvage

Purpose

Repair corrupted or inconsistent **data artifacts** (docs, packs, generated outputs) so that:

- references resolve again
- required fields exist
- validation can run
- the system returns to a consistent state

This is not a refactor mode; it is **surgical repair**.

Applies To

`allowed_target_types`: `axion_core` *(locked v1)*

Required Capabilities

- `node`
- `git` *(recommended for diff and audit)*
- Optional: tooling used by Axion validators

Runspec Inputs Used

- `target.target_id`

- `scope` (**must be explicit**; `paths` strongly recommended)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**must be true**)
- `metadata.notes` (must describe symptoms and repair intent)

Repair Inputs (Locked v1)

MM-11 requires an explicit repair plan or at minimum explicit repair intent. Because schema doesn't include structured fields, use:

- `metadata.notes` and/or
- `artifacts/repair_plan.md` (required in v1)

Required Companion Artifact (Repair Plan)

MM-11 must produce:

- `maintenance/runs/<run_id>/artifacts/repair_plan.md`

Required headers:

- `# Repair Plan`
- `## Symptoms`
- `## Target Scope`
- `## Repair Rules`
- `## Allowed Transformations`
- `## Verification`

Repair Rule Categories (Locked v1)

Repairs are only allowed within these categories (no freeform changes):

1. **Reference Fix**
    - fix broken pointers/links/IDs to existing targets
2. **Missing Required Fields**
    - insert missing required headers/fields with placeholders
3. **Normalization**
    - normalize formatting to canonical form (IDs, section headers)
4. **Orphan Handling**
    - mark or remove orphan references (depending on policy)
5. **Schema Alignment**
    - bring artifact shape up to current schema version (light repairs only; migrations belong in MM-09)

Any repair outside these categories must FAIL with pointers.

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm scope is explicit (paths)
    - Write `repair_plan.md`
    - Capture pre-repair diff state (git)
2. **Detect Issues**
    - Run a validator (if available) or perform filesystem checks to locate:
        - broken references

        - missing required fields
        - invalid IDs
    - Produce an issue list (structured):
        - `artifacts/issues.json` (recommended)

3. **Apply Repairs**
    - Apply allowed transformations only
    - Record each repair action:
        - what file
        - what change type
        - before/after snippet reference (optional)
4. **Post-Repair Validation**
    - Re-run validator (if available)
    - At minimum, re-scan for the same issues and confirm reduced count
5. **Diff Capture**
    - Write `artifacts/diff.patch`
6. **Write Report**
    - PASS if:
        - repairs applied and post-validation improved and/or passes
    - WARN if:
        - partial repair applied but remaining issues exist
    - FAIL if:
        - repairs cannot be applied or scope violation

Proof Requirements (Locked)

On PASS/WARN:

- `{ kind: "repair_complete", value: "salvage", evidence_path: "artifacts/repair_plan.md" }`

If diff generated:

- `{ kind: "diff_generated", value: "repair-diff", evidence_path: "artifacts/diff.patch" }`

Required `report.results[]` entries (minimum)

- `repair_plan` (PASS/FAIL)
- `issues_detected` (PASS/FAIL)
- `repairs_applied` (PASS/FAIL/WARN)
- `post_validation` (PASS/FAIL/WARN)
- `diff_capture` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/repair_plan.md`
- `artifacts/diff.patch`

Recommended:

- `artifacts/issues.json`
- `artifacts/repairs_applied.json`

Failure Semantics

- No explicit scope → FAIL
- No repair plan → FAIL
- Scope violation → FAIL
- Disallowed transformation attempted → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-11-data-repair.js`

Update defaults (recommended):

- `outputs.diff = true`
- `default_constraints.allow_writes = true`

Runner File Name

`maintenance/runners/mm-11-data-repair.js`

Runbook File Name

`maintenance/runbooks/MM-11.data-repair.md`

Example runspec (MM-11)

```
{
 "run_id": "placeholder",
```

"mode_id": "MM-11",
"target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
"scope": { "type": "paths", "paths": ["axion/domains/", "axion/registry/"], "components": [], "diff": { "from": "", "to": "" } },
"constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network": false },
"requested_outputs": { "diff": true, "snapshot": false, "changelog": false },
"expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json" }],
"metadata": { "env": "local", "requested_by": "agent", "notes": "Symptoms: broken references after merge; repair refs + missing required fields only." }
}

MM-10 Backcompat Validation

P3.1.7 — MM-10 Backcompat Validation (Mode Spec)

Mode ID

MM-10

Name

Backcompat Validation

Purpose

Validate that **older artifacts** (created under previous template/schema versions) are still:

- parseable
- acceptable under compatibility rules
- either fully valid or clearly flagged with deprecation warnings

This mode is read-only and is the “don’t break existing data” safety net.

Applies To

`allowed_target_types`: `axion_core | templates`

Required Capabilities

- `node`
- Optional: whatever validator/CLI Axion exposes for artifact validation
- `git` optional (metadata only)

Runspec Inputs Used

- `target.target_id`
- `scope` (paths recommended: point to legacy artifact folders)

- `constraints.allow_writes` (**must be false**)
- `metadata.notes` (must include where “legacy” artifacts live)

Compatibility Rules Source (Locked v1)

Backcompat validation requires an explicit rule source:

Priority order:

1. Axion migration/compatibility definitions (if present), e.g.:
    - `axion/migrations/`
    - `axion/config/*compat*`
2. Maintenance fallback (optional future):
    - `maintenance/registry/backcompat_rules.json`

If no compat rules are found → FAIL with pointers.

Legacy Artifact Set (Locked v1)

MM-10 must define what “legacy” means for the run:

- by folder (recommended): `scope.paths` includes `legacy/` or old pack folders
- by version marker inside artifacts (if your schema has it)

If no legacy set can be determined → WARN (not FAIL) in v1.

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=false`
    - Locate compatibility rules

    - Resolve legacy artifact set (from scope)

2. **Validate Legacy Artifacts**
   For each artifact:
    - parse it
    - validate under compatibility rules
    - categorize result:
        - PASS (fully compatible)
        - WARN (compatible but deprecated / missing non-critical fields)
        - FAIL (not compatible)
3. **Write Backcompat Report Artifact**
    - `artifacts/backcompat_report.json` containing:
        - total scanned
        - pass/warn/fail counts
        - list of failing artifacts with pointers + reason codes
4. **Write Report**
    - PASS if zero FAIL artifacts
    - WARN if FAIL exists but policy allows partial compatibility reporting (optional; v1 recommended FAIL if any FAIL)
    - FAIL if operational failures (missing rules, scan fail, etc.)

Proof Requirements (Locked)

On PASS:

- `{ kind: "backcompat_pass", value: "compat", evidence_path: "artifacts/backcompat_report.json" }`

On FAIL, proof may be empty; `results[]` must contain pointers.

Required `report.results[]` entries (minimum)

- `compat_rules_loaded` (PASS/FAIL)
- `legacy_set_resolved` (PASS/WARN/FAIL)
- `legacy_validation` (PASS/FAIL/WARN)
- `report_written` (PASS/FAIL)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/backcompat_report.json`

Optional:

- `artifacts/backcompat_summary.md`

Failure Semantics (Locked v1 recommendation)

- Any incompatible legacy artifact → `report.status = FAIL`
  (You can relax this later with a policy flag.)

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-10-backcompat-validate.js`

Update defaults (recommended):

- `default_constraints.allow_writes = false`

Runner File Name

`maintenance/runners/mm-10-backcompat-validate.js`

Runbook File Name

`maintenance/runbooks/MM-10.backcompat-validate.md`

Example runspec (MM-10)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-10",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/domains/", "legacy_artifacts/"], "components": [], "diff":
{ "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": false, "allow_network":
false },
 "requested_outputs": { "diff": false, "snapshot": false, "changelog": false },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Validate legacy artifacts under
backcompat rules; legacy paths include legacy_artifacts/." }
}
```

MM-09 Artifact Migration

P3.1.8 — MM-09 Artifact Migration (Mode Spec)

Mode ID

MM-09

Name

Artifact Migration

Purpose

Transform existing artifacts from an older schema/template version to the current version so that:

- validation passes under current rules
- future tooling operates on a single modern format
- backcompat burden is reduced over time

This is the “convert old → new” mode (as opposed to MM-10 which only checks compatibility).

Applies To

`allowed_target_types`: `axion_core | templates`

Required Capabilities

- `node`
- `git` *(strongly recommended; required in v1 for diff/audit and safe rollback)*
- Optional: Axion migration tooling, if available

Runspec Inputs Used

- `target.target_id`

- `scope` (**must be explicit**; paths strongly recommended)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**must be true**)
- `requested_outputs.snapshot` (recommended true before migration)
- `requested_outputs.changelog` (recommended true)
- `metadata.notes` (must include migration intent)

Migration Rules Source (Locked v1)

Migration must be driven by an explicit rules source. Priority order:

1. **Axion migrations directory** (preferred): `axion/migrations/`
2. **Maintenance migration rules** (optional future): `maintenance/registry/migration_rules.json`

If no migration rules exist → FAIL with pointers.

Required Companion Artifact (Migration Plan)

MM-09 must produce:

- `maintenance/runs/<run_id>/artifacts/migration_plan.md`

Required headers:

- `# Migration Plan`
- `## Source Version`
- `## Target Version`
- `## Scope`

- `## Migration Rules`
- `## Pre-Migration Snapshot`
- `## Verification`
- `## Rollback Plan`

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm explicit scope (paths)
    - Locate migration rules
    - Write `migration_plan.md`
    - (Recommended) create pre-migration snapshot:
        - `artifacts/pre_migration_snapshot.zip` (or require MM-16 beforehand)
2. **Detect Candidates**
    - Scan scope for artifacts that match “source version” characteristics:
        - version field inside docs (if exists)
        - folder location
        - schema signature
    - Write `artifacts/migration_candidates.json` (recommended)
3. **Apply Migration**
    - Execute migration transforms deterministically:

        - field renames
        - section/header insertion
        - ID normalization
        - structure changes per rules
    - Record actions:
        - `artifacts/migration_actions.json` (recommended)

4. **Post-Migration Validation**
    - Run current validator/verify on migrated artifacts:
        - ideally invoke Axion validation tooling
    - If validation fails → FAIL (or WARN in relaxed mode later)
5. **Diff Capture**
    - Write `artifacts/diff.patch`
6. **Changelog**
    - If requested: write `artifacts/changelog.md` migration entry
7. **Write Report**
    - PASS only if migration + post-validation succeed

Proof Requirements (Locked)

On PASS:

- `{ kind: "migration_applied", value: "artifacts", evidence_path: "artifacts/migration_plan.md" }`
- `{ kind: "verify_pass", value: "post-migration", evidence_path: "runlog.md" }`

Also required:

- `{ kind: "diff_generated", value: "migration-diff", evidence_path: "artifacts/diff.patch" }`

Required `report.results[]` entries (minimum)

- `migration_rules_loaded` (PASS/FAIL)
- `candidates_detected` (PASS/FAIL/WARN)
- `migration_applied` (PASS/FAIL)
- `post_migration_validation` (PASS/FAIL)
- `diff_capture` (PASS/FAIL)
- `snapshot` (PASS/FAIL/WARN if requested)
- `changelog_update` (PASS/FAIL/WARN if requested)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`
- `artifacts/migration_plan.md`
- `artifacts/diff.patch`

Recommended:

- `artifacts/pre_migration_snapshot.zip`

- `artifacts/migration_candidates.json`
- `artifacts/migration_actions.json`
- `artifacts/changelog.md`

Failure Semantics

- Missing rules → FAIL
- No explicit scope → FAIL
- Post-migration validation fails → FAIL
- Partial migration should be WARN only if explicitly allowed by policy (v1 default: FAIL)

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-09-migrate-artifacts.js`

Update defaults (recommended):

- `outputs.diff = true`
- `outputs.snapshot = true`
- `outputs.changelog = true`
- `default_constraints.allow_writes = true`

Runner File Name

`maintenance/runners/mm-09-migrate-artifacts.js`

Runbook File Name

`maintenance/runbooks/MM-09.migrate-artifacts.md`

Example runspec (MM-09)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-09",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/domains/", "legacy_artifacts/"], "components": [], "diff":
{ "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": true, "changelog": true },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Migrate legacy artifacts to current
schema; source=v1, target=v2." }
}
```

MM-07 Deprecation Rollout

P3.1.9 — MM-07 Deprecation Rollout (Mode Spec)

Mode ID

MM-07

Name

Deprecation Rollout

Purpose

Introduce a controlled deprecation cycle by:

- marking items as deprecated
- emitting warnings (in validation/UI if applicable)
- defining removal timeline/version
- ensuring docs and registries reflect the deprecation

This mode helps you phase out templates/artifacts/rules safely before removal.

Applies To

`allowed_target_types`: `axion_core | templates`

Required Capabilities

- `node`
- `git` *(strongly recommended for diff + audit)*
- Optional: whichever system emits warnings (validator/UI)

Runspec Inputs Used

- `target.target_id`

- `scope` (**must be explicit**; paths/components)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**must be true**)
- `requested_outputs.changelog` (**must be true**)
- `metadata.notes` (must include deprecation intent)

Deprecation Inputs (Locked v1)

Deprecation must explicitly declare:

- what is being deprecated (IDs/paths)
- replacement (if any)
- effective date/version
- removal date/version

Because schema doesn't include structured fields, capture in:

- `artifacts/deprecation_plan.md` (required)

Required Companion Artifact (Deprecation Plan)

MM-07 must produce:

- `maintenance/runs/<run_id>/artifacts/deprecation_plan.md`

Required headers:

- `# Deprecation Plan`
- `## Items Deprecated`

- `## Replacement / Migration Path`
- `## Effective Version/Date`
- `## Removal Version/Date`
- `## Required Warnings`
- `## Verification`

Deprecation Mechanisms (Locked v1 set)

Deprecation is enacted by editing one or more of:

1. **Registries**
    - mark entries with `deprecated: true`
    - add `deprecated_reason`, `deprecated_since`, `removal_target`
2. **Docs**
    - update system docs describing deprecated items
3. **Validation/Warn Rules** (optional)
    - add warning outputs for deprecated usage
4. **UI Labels** (optional)
    - UI shows “Deprecated” tag

v1 rule: at minimum, registries + docs must be updated.

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`

    - Confirm explicit scope
    - Write `deprecation_plan.md`
    - Capture pre-change state (git)
2. **Apply Deprecation Flags**
    - Update registry entries to include deprecation metadata
    - Update docs to reflect deprecation and timeline
3. **Warning Hook (optional v1)**
    - If validator/UI has a warning system, add/verify warning emission
4. **Diff Capture**
    - Write `artifacts/diff.patch`
5. **Changelog Update**
    - Write `artifacts/changelog.md` entry describing deprecation
6. **Verification**
    - Run `targets.json.runtime_contract.verify` (recommended)
    - Ensure no breaking behavior introduced (deprecation should be non-breaking)
7. **Write Report**
    - PASS if:
        - registries updated
        - docs updated
        - diff + changelog exist
        - verify passes (if run)

Proof Requirements (Locked)

On PASS:

- `{ kind: "deprecation_applied", value: "deprecations", evidence_path: "artifacts/deprecation_plan.md" }`
- `{ kind: "diff_generated", value: "deprecation-diff", evidence_path: "artifacts/diff.patch" }`
- `{ kind: "changelog_written", value: "deprecation-changelog", evidence_path: "artifacts/changelog.md" }`

If verify is executed:

- `{ kind: "verify_pass", value: "post-deprecation", evidence_path: "runlog.md" }`

Required `report.results[]` entries (minimum)

- `deprecation_plan` (PASS/FAIL)
- `registry_updated` (PASS/FAIL)
- `docs_updated` (PASS/FAIL)
- `diff_capture` (PASS/FAIL)
- `changelog_update` (PASS/FAIL)
- `verify` (PASS/FAIL/WARN if verify not run)

Outputs

Always:

- `runspec.json`

- `runlog.md`
- `report.json`
- `artifacts/deprecation_plan.md`
- `artifacts/diff.patch`
- `artifacts/changelog.md`

Optional:

- `artifacts/warnings_demo.md` (evidence of warnings)

Failure Semantics

- No plan → FAIL
- No registry change → FAIL
- No docs update → FAIL
- Deprecation must not break verification; if it does → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-07-deprecate.js`

Update defaults (recommended):

- `outputs.diff = true`
- `outputs.changelog = true`
- `default_constraints.allow_writes = true`

Runner File Name

`maintenance/runners/mm-07-deprecate.js`

Runbook File Name

`maintenance/runbooks/MM-07.deprecate.md`

Example runspec (MM-07)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-07",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/registry/", "axion/templates/"], "components": [], "diff":
{ "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": false, "changelog": true },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "local", "requested_by": "agent", "notes": "Deprecate template set v1;
replacement=v2; removal in v3.0.0." }
}
```

MM-08 Breaking Upgrade

P3.1.10 — MM-08 Breaking Upgrade (Major Version) (Mode Spec)

Mode ID

MM-08

Name

Breaking Upgrade (Major Version)

Purpose

Execute an **intentional breaking change** to Axion's contracts (templates/schemas/registries/engine behaviors) with:

- explicit migration plan
- explicit compatibility decision
- major version bump discipline
- strict verification + migration validation

This is the most controlled and strict upgrade mode.

Applies To

`allowed_target_types`: `axion_core | templates` *(locked v1)*

Required Capabilities

- `node`
- `git` *(required; major upgrades must be auditable and revertible)*
- `npm` *(if build/verify relies on npm)*
- Optional: migration tooling

Runspec Inputs Used

- `target.target_id`
- `scope` (**must be explicit**; paths/components)
- `constraints.allow_writes` (**must be true**)
- `requested_outputs.diff` (**must be true**)
- `requested_outputs.snapshot` (**must be true**)
- `requested_outputs.changelog` (**must be true**)
- `metadata.notes` (must state major version intent)

Required Companion Artifacts (Locked)

MM-08 requires these artifacts in the run folder:

1. `artifacts/major_upgrade_plan.md`
2. `artifacts/migration_plan.md`
3. `artifacts/compatibility_statement.md`

If any are missing → FAIL.

1) `major_upgrade_plan.md` required headers

- `# Major Upgrade Plan`
- `## Summary`
- `## Breaking Changes`
- `## Scope`

- `## Dependency Closure`
- `## Verification`

2) `migration_plan.md` required headers

- `# Migration Plan`
- `## Source Version`
- `## Target Version`
- `## Migration Rules`
- `## Migration Steps`
- `## Validation`
- `## Rollback`

3) `compatibility_statement.md` required headers

- `# Compatibility Statement`
- `## What Breaks`
- `## Who Is Impacted`
- `## Backcompat Policy`
- `## Deprecation Timeline (if any)`

Major Version Discipline (Locked v1)

MM-08 must explicitly set/record:

- source version (current)

- target major version (new major)

In v1, this is recorded in:

- `compatibility_statement.md`
- `major_upgrade_plan.md`
- `artifacts/changelog.md`
- `artifacts/release_manifest.json` (if packaging occurs)

Steps (Deterministic)

1. **Preflight**
    - Confirm `allow_writes=true`
    - Confirm explicit scope
    - Confirm required artifacts exist (or generate if your workflow allows the agent to write them first)
    - Capture pre-upgrade snapshot:
        - `artifacts/pre_upgrade_snapshot.zip`
2. **Apply Breaking Patch Set**
    - Apply changes strictly in-scope
    - Changes typically include:
        - template schema changes
        - registry changes
        - engine changes
        - removal/rename of artifacts

- Record patch actions in runlog

3. **Execute Migration**
   - Run migration steps per `migration_plan.md`
   - If migration tooling exists, invoke it; otherwise apply scripted transforms
   - Record migrated file counts and any failures
4. **Post-Upgrade Verification**
   - Run:
     - `targets.json.runtime_contract.verify`
     - and if defined: `runtime_contract.build`
   - Must pass for PASS
5. **Diff Capture**
   - Write `artifacts/diff.patch`
6. **Changelog Update**
   - Write `artifacts/changelog.md` (must include breaking changes + migration notes)
7. **Post-Upgrade Snapshot**
   - Write `artifacts/post_upgrade_snapshot.zip` (optional but recommended)
8. **Write Report**
   - PASS only if:
     - migration executed successfully
     - post-upgrade verify passes

    - diff/snapshot/changelog exist

Proof Requirements (Locked)

On PASS:

- `{ kind: "migration_validated", value: "major-upgrade", evidence_path: "artifacts/migration_plan.md" }`
- `{ kind: "verify_pass", value: "post-major-upgrade", evidence_path: "runlog.md" }`
- `{ kind: "diff_generated", value: "major-upgrade-diff", evidence_path: "artifacts/diff.patch" }`
- `{ kind: "snapshot_exists", value: "pre-upgrade", evidence_path: "artifacts/pre_upgrade_snapshot.zip" }`
- `{ kind: "changelog_written", value: "major-upgrade-changelog", evidence_path: "artifacts/changelog.md" }`

Required `report.results[]` entries (minimum)

- `plans_present` (PASS/FAIL)
- `pre_upgrade_snapshot` (PASS/FAIL)
- `patch_applied` (PASS/FAIL)
- `migration_executed` (PASS/FAIL)
- `post_upgrade_verify` (PASS/FAIL)
- `diff_capture` (PASS/FAIL)
- `changelog_update` (PASS/FAIL)
- `post_upgrade_snapshot` (PASS/FAIL/WARN if optional)

Outputs

Always:

- `runspec.json`
- `runlog.md`
- `report.json`

Required artifacts:

- `artifacts/major_upgrade_plan.md`
- `artifacts/migration_plan.md`
- `artifacts/compatibility_statement.md`
- `artifacts/pre_upgrade_snapshot.zip`
- `artifacts/diff.patch`
- `artifacts/changelog.md`

Optional:

- `artifacts/post_upgrade_snapshot.zip`

Failure Semantics (Strict)

- Missing any required plan → FAIL
- Migration fails → FAIL
- Verification fails → FAIL
- Missing diff/snapshot/changelog → FAIL

Mode Registry Entry (switch stub to real)

Update runner to:

- `runners/mm-08-major-upgrade.js`

Update defaults (recommended):

- `outputs.diff = true`
- `outputs.snapshot = true`
- `outputs.changelog = true`
- `default_constraints.allow_writes = true`

Runner File Name

`maintenance/runners/mm-08-major-upgrade.js`

Runbook File Name

`maintenance/runbooks/MM-08.major-upgrade.md`

Example runspec (MM-08)


```
{
 "run_id": "placeholder",
 "mode_id": "MM-08",
 "target": { "target_id": "axion-core", "root": ".", "type": "axion_core" },
 "scope": { "type": "paths", "paths": ["axion/templates/", "axion/registry/", "axion/config/"],
"components": [], "diff": { "from": "", "to": "" } },
 "constraints": { "stop_on_fail": true, "locked_paths": [], "allow_writes": true, "allow_network":
false },
 "requested_outputs": { "diff": true, "snapshot": true, "changelog": true },
 "expected_proof": [{ "kind": "required_from_modes_json", "value": "required_from_modes_json"
}],
 "metadata": { "env": "prod", "requested_by": "agent", "notes": "Major upgrade: v2.0.0; breaking
template schema + registry model; migration required." }
}
```

Mode Registry Governance

Mode Registry Governance (Outline)

1) Purpose

- Define how `maintenance/registry/modes.json` is managed so mode execution stays deterministic.
- Prevent drift: no silent changes to proofs, constraints, or runner mappings.

2) Scope

- Applies to:
  - `maintenance/registry/modes.json`
  - `maintenance/registry/policies.json`
  - `maintenance/schemas/*`
  - runner entrypoints under `maintenance/runners/`
- Does not cover: Axion internal registries (`/axion/registry/*`)

3) Source of Truth Order

- Order of authority when fields conflict:
  1. `policies.json` (global enforcement + override rules)
  2. `modes.json` (mode definition + required proofs + defaults)
  3. `runspec.json` (run-specific inputs, only where overrides are allowed)

4) Mode Lifecycle States

- `stub` → `implemented` → `deprecated` → `retired`

- Definitions and allowed transitions.

5) Mode Definition Contract (modes.json schema)

- Required fields per mode:
  - mode_id, name, description
  - runner.entrypoint + runner.kind
  - allowed_target_types
  - required_capabilities
  - default_constraints
  - required_proof
  - outputs
- Optional fields:
  - severity (normal/strict)
  - supports_scope_types (all/paths/components/diff_range)
  - warnings_as_fail (bool)

6) Stub → Implemented Promotion Rules

- What must exist before promotion:
  - runner file exists and is executable
  - runbook exists
  - produces runspec/runlog/report with schema validity
  - proof kinds match `required_proof`
- How to mark promotion in modes.json (state field or entrypoint change).

7) Proof Governance

- Locked proof rules:
  - `required_proof` cannot be overridden by runspec (policy default)
  - proof kind vocabulary (allowed values list)
  - evidence_path rules (must be inside run folder)
- Proof change policy:
  - when a proof requirement can change
  - version bump requirements when it changes

8) Constraint Governance

- What constraints are mode-owned vs run-owned
- Locked defaults:
  - allow_writes default per mode
  - allow_network default per mode
  - locked_paths per mode + global locks
- Override policy (what runspec may override)

9) Runner Mapping & Compatibility

- entrypoint naming conventions
- runner kind rules (node/shell/python)
- backward compatibility for renamed runner files
- “not implemented” handling standard

10) Versioning Policy

- `modes.json` version field rules
- Schema versioning rules
- When changes are patch/minor/major

11) Change Control Process

- Required artifacts for changes:
    - registry change summary
    - diff patch
    - verification run (MM-01 + affected mode)
- Review/approval gates (if you want them)

12) Auditing & Traceability

- How to record:
    - who changed what
    - when a mode was promoted
    - what proof/constraints changed
- Recommended: add a `maintenance/registry/REGISTRY_CHANGELOG.md`

13) Failure Handling & Emergency Edits

- What to do if modes registry is broken
- Safe recovery steps (MM-21 or manual minimal restore)
- Emergency override rules (if allowed)

14) Appendices

- A) Mode state definitions (stub/implemented/deprecated/retired)
- B) Proof kind vocabulary list (v1)
- C) Example modes.json entry (implemented)
- D) Example promotion checklist

Source of Truth Order

Source of Truth Order

Because it defines the authority chain for everything else:

- policies vs modes vs runspec
- what can/can’t be overridden
- how the dispatcher must behave

Once Section 3 is locked, every other section becomes straightforward and consistent (lifecycle, proofs, constraints, versioning, change control).

Start order (recommended)

1. **Section 3 — Source of Truth Order** (lock the authority model)
2. **Section 5 — Mode Definition Contract** (what a mode entry must contain)
3. **Section 7 — Proof Governance** (what “proof” means and how it’s locked)
4. **Section 8 — Constraint Governance** (safety controls + overrides)
5. Then the rest (lifecycle, promotion rules, versioning, change control, audit)

Tab 45

Agent vs Control Plane

Agent vs Control Plane (Outline)

1) Purpose

- Define the division of labor between the **Agent (operator + judgment)** and the **Maintenance Control Plane (mechanics + enforcement)**.
- Reduce token usage by pushing repeatable work into automation without limiting decision quality.

---

2) Core Principle

- **Control Plane owns mechanics.**
- **Agent owns judgment.**
- Agent speaks only at **decision points**; everything else is runner output.

---

3) Control Plane Responsibilities (What it owns)

3.1 Deterministic Execution

- Mode step graphs (per MM-xx)
- Runner invocation and lifecycle
- Non-interactive execution (CI-safe)

3.2 Enforcement & Safety

- Constraint defaults (allow_writes, allow_network, stop_on_fail)
- Locked paths (global + per-mode)
- Override permissions (what runspec may override)
- Post-run enforcement (git diff / scope checks)

3.3 Definitions of Done

- Required proof sets per mode
- PASS/FAIL/WARN rules
- Failure class codes (standardized failure reasons)

3.4 Target & Environment Normalization

- Target registry + runtime contracts (verify/build/ci/rebuild commands)
- Capability detection (node/npm/git/etc.)

- Standard exclusions for snapshots/diffs (e.g., .git, node_modules, runs)

3.5 Artifact Production

- Run folder creation
- runspec finalization
- runlog writing
- report.json generation
- optional artifacts (diff/snapshot/changelog/plans)

---

4) Agent Responsibilities (What it owns)

4.1 Mode Selection

- Choose the correct `mode_id` (MM-xx)

4.2 Scope Selection

- Choose scope type (all/paths/components/diff_range)
- Prefer components → paths mapping when available

4.3 Limited Overrides (Only if policy allows)

- Request optional outputs (diff/snapshot/changelog)
- Provide notes/context

4.4 Decision-Point Reasoning

- Interpret report.json failures (MM-20)
- Create dependency closure plans (MM-05/MM-08)
- Choose recovery actions (MM-21)
- Decide next run (loop control)

4.5 Human Communication

- Summarize strictly from `report.json`
- Propose next action in 1–3 steps (no long narratives unless requested)

---

5) Shared Interface (Handshake Contract)

5.1 Inputs (Agent → Control Plane)

- `runspec.json` (minimal intent)
- Dispatcher invocation command

5.2 Outputs (Control Plane → Agent)

- `runlog.md`
- `report.json` (source of truth)
- artifacts folder (diff/snapshot/changelog/plan docs)

5.3 The Loop

1. Agent creates runspec
2. Agent runs dispatcher
3. Control plane produces outputs
4. Agent reads report + decides next mode/scope

---

6) Reasoning Boundaries (Token Control)

6.1 What is NOT reasoning (should be automated)

- executing commands
- collecting logs
- generating reports/artifacts
- enforcing constraints

6.2 What IS reasoning (agent-only, but minimized)

- selecting mode/scope
- interpreting ambiguous failures
- upgrade/migration planning
- recovery choice

6.3 Reasoning budgets by mode (optional extension)

- reasoning_level: none/low/medium/high per MM-xx
- response format restrictions for low reasoning modes

---

7) Mode-by-Mode Responsibility Matrix (Table)

For each MM-xx:

- Control plane: steps + proofs + enforcement

- Agent: decisions required (if any)
- Required artifacts

---

8) Governance & Change Control

- What changes require updating control plane registries
- How to prevent agents from “making new procedures”
- Promotion rules (stub → implemented runner)

---

9) Appendices

- A) Failure class vocabulary
- B) Components → paths mapping
- C) Sample minimal runspec templates
- D) Sample “agent response formats” for low-token modes

File Tree

```
/maintenance
  /docs
    Mode_Execution_Contract.md
    Mode_Run_Artifact_Standards.md
    Mode_Registry_Governance.md

  /prompts
    MAINT_RUN.prompt.md
    MAINT_UPGRADE.prompt.md
    MAINT_TRIAGE.prompt.md
    MAINT_RECOVERY.prompt.md
    README.md

  /schemas
    runspec.schema.json
    report.schema.json

  /registry
    modes.json
    targets.json
    policies.json
    REGISTRY_CHANGELOG.md

  /runbooks
    README.md
    MM-01.health-check.md
    MM-02.full-verify.md
    MM-03.coverage-audit.md
    MM-04.drift-detect.md
    MM-05.planned-upgrade.md
    MM-06.hotfix.md
    MM-07.deprecate.md
    MM-08.major-upgrade.md
    MM-09.migrate-artifacts.md
    MM-10.backcompat-validate.md
    MM-11.data-repair.md
    MM-12.rebuild-indexes.md
    MM-13.registry-cleanup.md
    MM-14.release-package.md
    MM-15.rollback.md
    MM-16.snapshot.md
    MM-17.deps-refresh.md
    MM-18.workspace-compat.md
    MM-19.ci-sanity.md
```

```
  MM-20.triage.md
  MM-21.recovery.md

/runners
  dispatcher.js
  mm-stub-not-implemented.js
  mm-01-health-check.js
  mm-02-full-verify.js
  mm-03-coverage-audit.js
  mm-04-drift-detect.js
  mm-05-planned-upgrade.js
  mm-06-hotfix.js
  mm-07-deprecate.js
  mm-08-major-upgrade.js
  mm-09-migrate-artifacts.js
  mm-10-backcompat-validate.js
  mm-11-data-repair.js
  mm-12-rebuild-indexes.js
  mm-13-registry-cleanup.js
  mm-14-release-package.js
  mm-15-rollback.js
  mm-16-snapshot.js
  mm-17-deps-refresh.js
  mm-18-workspace-compat.js
  mm-19-ci-sanity.js
  mm-20-triage.js
  mm-21-recovery.js

/runs
  (generated at runtime; ignored by git)
  /<run_id>/
    runspec.json
    runlog.md
    report.json
    /artifacts
      diff.patch
      snapshot.zip
      changelog.md
      *.md
      *.json
```

maintenance/docs/Mode_Execution_Contract.md

Mode Execution Contract

1) Purpose
Define the deterministic execution rules for Axion Maintenance modes (MM-01..MM-21) so:
- runs are repeatable and auditable
- agents do not improvise procedures
- outputs and “done” criteria are machine-verifiable

This contract governs the external maintenance control plane (the `/maintenance` folder), not Axion internal control planes.

---

2) Core Concepts

2.1 Mode Run
A **Mode Run** is the only valid unit of maintenance work. A run is defined by:
- a `mode_id` (MM-xx)
- a `target_id`
- a `runspec.json` (intent + boundaries)
- deterministic execution by a mode runner
- standard run artifacts (`runspec/runlog/report`)

2.2 Hybrid Execution Model (A + B)
- **Prompt layer (A):** operator/agent creates a runspec and triggers the dispatcher.
- **Automation layer (B):** dispatcher + runners perform deterministic steps and produce proof artifacts.

The agent is the operator and decision-maker at specific decision points, but not the process authority.

---

3) Source of Truth Order (Authority)
When values conflict, resolve in this order:

1) `maintenance/registry/policies.json` (global enforcement + override permissions)
2) `maintenance/registry/modes.json` (mode-level contract: runner, proofs, defaults)
3) `runspec.json` (run-level intent, only where overrides are permitted)

---

4) Entrypoint (Dispatcher)
All mode runs must be invoked through a single dispatcher interface:

- `maintenance run <mode_id> --spec <path_to_runspec>`

Implementation may be Node/Shell/Python, but the interface and semantics are fixed.

---

5) Dispatcher Responsibilities (Required)

5.1 Validate Input
Dispatcher must:
- load the pre-run spec file
- validate it against `maintenance/schemas/runspec.schema.json`
- fail fast if invalid

5.2 Resolve Mode Definition
Dispatcher must load `maintenance/registry/modes.json` and resolve `mode_id`:
- if missing → FAIL

5.3 Resolve Target Definition
Dispatcher must load `maintenance/registry/targets.json` and resolve `target_id`:
- if missing → FAIL
- target registry is authoritative for root/type/runtime_contract

5.4 Finalize Run ID
Dispatcher generates the final `run_id` and overwrites the spec:
- `run_id` inside user spec is treated as a placeholder

5.5 Create Run Directory
Dispatcher must create:
- `maintenance/runs/<run_id>/`
- `maintenance/runs/<run_id>/artifacts/`

5.6 Write Final Runspec
Dispatcher must write the finalized spec to:
- `maintenance/runs/<run_id>/runspec.json`

This is the run's source of truth.

5.7 Merge Constraints
Dispatcher must compute effective constraints:
- start with `policies.defaults`
- merge `modes.default_constraints`
- merge `runspec.constraints` only if policy allows overrides

Global locked paths always apply.

5.8 Enforce Required Proof (Policy Default)
- `modes.required_proof` is the minimum proof set for a mode.
- If `policies.overrides.allow_runspec_override_required_proof = false`:
  - dispatcher overwrites `runspec.expected_proof` with `modes.required_proof`

5.9 Capability Checks
Dispatcher must verify `modes.required_capabilities` are present.
If missing → FAIL fast.

5.10 Execute Runner
Dispatcher invokes the runner defined in `modes.json.runner.entrypoint`.
It must pass at least:
- MODE_ID
- RUN_ID
- RUN_DIR
- TARGET_ID
- TARGET_ROOT

5.11 Guarantee Standard Artifacts
Even on failures, dispatcher must guarantee:
- `runlog.md` exists
- `report.json` exists (schema-valid)
If the runner fails to write report, dispatcher must write a minimal FAIL report.

5.12 Exit Code Semantics
Dispatcher exit code is derived from report status (preferred) or runner exit code (minimum):
- PASS → 0
- FAIL → 1
- WARN → 2 (optional; allowed)

---

6) Runner Responsibilities (Required)

6.1 Deterministic Steps
Runners must follow the mode's runbook/step graph and must be non-interactive.

6.2 Standard Outputs
Runners must write:
- `runlog.md`
- `report.json`

Optional artifacts go under:
- `maintenance/runs/<run_id>/artifacts/`

6.3 Report Truth
Runners must not claim PASS unless required proofs are satisfied and recorded in `report.proof`.

6.4 Respect Constraints
Runners must respect:
- allow_writes
- locked_paths
- allow_network

If a runner needs writes but allow_writes=false, it must FAIL.

---

7) Enforcement Rules

7.1 allow_writes=false (MVP)
When allow_writes=false:
- only writes permitted are under `maintenance/runs/<run_id>/`
- dispatcher performs post-run enforcement (MVP: git status/diff)
- if disallowed changes exist → force FAIL

7.2 locked_paths
Global locked paths from policies must always be enforced.
Per-mode locked paths are additive.

7.3 stop_on_fail
Default stop-on-fail = true.
A runner may implement “continue-on-fail” only if the mode explicitly allows it.

---

8) Failure Handling
Dispatcher must fail fast for:
- invalid runspec schema
- missing mode_id/target_id
- missing required capabilities
Runner failures must produce:
- a FAIL report with pointers and a human-readable summary in runlog

---

9) Audit Trail
A Mode Run is auditable if:
- run directory exists
- runspec/runlog/report exist
- report contains proofs and pointers
- (optional) diff/snapshot artifacts exist when requested

---

10) Non-Goals
- This contract does not define Axion internal build logic.
- This contract does not dictate code style; it dictates execution semantics and artifacts.

maintenance/docs/Mode_Run_Artifact_Standards.md

Mode Run Artifact Standards

1) Purpose
Standardize run outputs so:
- every run is comparable
- every run is auditable
- PASS/FAIL is machine-verifiable
- agents summarize from structured evidence instead of narrative

---

2) Run Directory Layout (Locked)
Every run writes to:

- `maintenance/runs/<run_id>/`
  - `runspec.json`
  - `runlog.md`
  - `report.json`
  - `artifacts/` (optional outputs)

`maintenance/runs/` is generated output and should be ignored by git.

---

3) Required Artifacts (Locked)
Every run must produce:

1) `runspec.json` — intent + boundaries (finalized by dispatcher)
2) `runlog.md` — human-readable command trace + summaries
3) `report.json` — machine-readable truth of the run

A run is invalid if any is missing.

---

4) runspec.json (Contract)
- Must validate against `maintenance/schemas/runspec.schema.json`
- Required fields:
  - run_id
  - mode_id
  - target
  - scope
  - constraints
  - requested_outputs

  - expected_proof
  - metadata

4.1 Runspec Principles
- Runspec declares intent and boundaries.
- Dispatcher finalizes run_id and enforces mode_id and required proofs.

---

5) runlog.md (Contract)
`runlog.md` must be readable and minimally structured.

5.1 Required Sections (Recommended order)
- Run header (run_id, mode_id, target_id, timestamps)
- Preconditions (capabilities detected, versions)
- Steps executed (timestamped)
- Command output summaries
- Artifacts written (paths)
- Result (PASS/FAIL/WARN)

Runners may add more detail, but must keep the runlog concise and navigable.

---

6) report.json (Contract)
- Must validate against `maintenance/schemas/report.schema.json`
- Required fields:
  - run_id
  - mode_id
  - status (PASS|FAIL|WARN)
  - target (target_id, commit, branch)
  - started_at / finished_at
  - results[] (at least one)
  - proof[] (may be empty on FAIL)
  - artifacts (paths)

6.1 results[] Structure
Each result item must include:
- check_id (stable identifier)
- status (PASS|FAIL|WARN)
- summary (short text)
- pointers[] (0+)

6.2 Pointer Contract

Each pointer must include:
- path
- line (integer, 0 allowed)
- message

Pointers are best-effort; if unavailable, include at least path-level pointers.

---

7) Proof Contract
Proof items are the machine-verifiable “definition of done.”

Each proof item includes:
- kind
- value
- evidence_path

7.1 Required Proof
`modes.json.required_proof` is the minimum proof set.
Policy default: runspec cannot override required proof.

---

8) Optional Artifacts (artifacts/)
Optional artifacts must live under:
- `maintenance/runs/<run_id>/artifacts/`

8.1 Standard Optional Files
- `diff.patch` (preferred) or `diff.md`
- `snapshot.zip` (+ `snapshot_manifest.json`)
- `changelog.md`
- plan docs (upgrade/migration/deprecation/recovery)

8.2 Naming Rules
Use stable names so tools can find them:
- `diff.patch`
- `snapshot.zip`
- `snapshot_manifest.json`
- `changelog.md`

Plan docs should be named by mode intent:
- `upgrade_plan.md`
- `hotfix_plan.md`
- `migration_plan.md`

- `deprecation_plan.md`
- `release_plan.md`
- `rollback_plan.md`
- `recovery_plan.md`
- `triage.md`

---

9) Artifact Truth Rules (Locked)
- `report.json` is the truth source for status.
- Agent summaries must be derived from `report.json` fields, not from speculation.
- If optional artifacts are requested, runners must either produce them or record FAIL/WARN with pointers explaining why.

---

10) Non-Goals
- This doc does not define the content of Axion artifacts/templates.
- This doc defines the maintenance run artifact standards only.

maintenance/docs/Mode_Registry_Governance.md

Mode Registry Governance

1) Purpose
Define governance for:
- `maintenance/registry/modes.json`
- `maintenance/registry/targets.json`
- `maintenance/registry/policies.json`

So that maintenance behavior is deterministic, auditable, and resistant to agent drift.

---

2) Scope
Governs:
- maintenance registries
- schemas under `maintenance/schemas/`
- runner entrypoints under `maintenance/runners/`

Does not govern:
- Axion internal registries under `/axion/*`

---

3) Source of Truth Order (Locked)
1) policies.json
2) modes.json
3) runspec.json

(See Mode Execution Contract for resolution semantics.)

---

4) Mode Lifecycle States
A mode progresses through:

1) **stub**
- runner points to `mm-stub-not-implemented`
- mode exists for completeness but is not usable for real work

2) **implemented**
- runner exists and executes deterministically
- produces required artifacts and proofs

3) **deprecated**

- still runnable, but emits warnings and has an end-of-life target

4) **retired**
- removed from modes.json (or kept but blocked), not executable

---

5) Mode Definition Contract (modes.json)
Each mode entry must include:
- mode_id (MM-01..MM-21)
- name
- description
- runner (entrypoint, kind)
- allowed_target_types
- required_capabilities
- default_constraints
- required_proof
- outputs

The mode entry is the contract. Runners must conform.

---

6) Stub → Implemented Promotion Rules (Locked)
A mode may be promoted to implemented only when all are true:

1) Runner file exists at `maintenance/runners/<runner>.js` (or equivalent kind)
2) Runbook exists at `maintenance/runbooks/<MM-xx>.*.md`
3) Running the mode produces:
   - runspec.json
   - runlog.md
   - report.json (schema-valid)
4) report includes required proof kinds defined in `modes.json.required_proof`
5) Constraints are respected (allow_writes/locked_paths/allow_network)

Promotion action:
- update `modes.json.runner.entrypoint` away from stub to the real runner

---

7) Proof Governance (Locked)
7.1 Required Proof
- required_proof defines the minimum “definition of done.”
- runspec cannot remove or replace required proof under default policy.

7.2 Proof Vocabulary Discipline
Proof kinds should be stable and reused, not invented per run.
Examples:
- command_pass
- verify_pass
- snapshot_exists
- diff_generated
- changelog_written
- registry_build_pass
- workspace_contract_pass
- triage_report_generated
- recovery_actions_applied
- migration_applied
- migration_validated
- backcompat_pass
- deps_updated
- ci_pass
- registry_clean
- repair_complete
- deprecation_applied
- package_exists
- rollback_complete

7.3 Changing Proof Requirements
Changing required_proof is a governance event:
- must be recorded in `REGISTRY_CHANGELOG.md`
- should trigger a modes.json version bump (see Versioning Policy)

---

8) Constraint Governance (Locked)
8.1 Where constraints live
- policies.json: global defaults and override permissions
- modes.json: per-mode default constraints
- runspec: run-time overrides only if policy allows

8.2 Global locks
Global locked paths always apply and cannot be removed by runspec.

8.3 Changing default constraints
Changing default_constraints is a governance event:
- record in REGISTRY_CHANGELOG.md
- verify affected modes still behave safely

---

9) Runner Mapping Governance
9.1 Entrypoint naming
Runner filenames should be stable, mode-prefixed:
- `mm-02-full-verify.js`, etc.

9.2 Breaking runner renames
If runner files are renamed:
- update modes.json accordingly
- record in REGISTRY_CHANGELOG.md
- run MM-01 + affected mode as verification

---

10) targets.json Governance
targets.json is authoritative for:
- target roots
- runtime_contract commands (verify/build/ci/rebuild)

Any change to runtime_contract commands must be recorded in REGISTRY_CHANGELOG.md.

---

11) policies.json Governance
policies.json is authoritative for:
- override permissions
- global locks
- default stop-on-fail/network policy
- write enforcement rules

Changing override permissions is a high-impact change:
- record in REGISTRY_CHANGELOG.md
- re-verify foundational modes (MM-01/MM-02/MM-18 at minimum)

---

12) Versioning Policy
Each registry file includes a top-level `version`.
Rules:
- Patch: typo fixes, description changes, non-behavior metadata
- Minor: adding modes/targets, adding optional fields, adding new proof kinds (non-breaking)

- Major: changing required_proof semantics, changing override permissions, changing schema contracts

Schema versions should be bumped when required fields/structures change.

---

13) Change Control Process (Minimum)
For any change to policies/modes/targets/schemas:
1) make the change
2) update `REGISTRY_CHANGELOG.md`
3) run MM-01 (health) and any directly affected modes
4) record the run_id(s) in REGISTRY_CHANGELOG.md

---

14) Auditing & Traceability
Maintain:
- `maintenance/registry/REGISTRY_CHANGELOG.md`
Each entry should include:
- date
- files changed
- summary
- why
- verification run_ids

---

15) Emergency Edits
If registries are broken:
- restore from last known good commit or snapshot
- prioritize making MM-01 runnable first
- then rebuild/repair other registry fields

maintenance/docs/Agent_vs_ControlPlane.md

Agent vs Control Plane Operating Contract

1) Purpose
Define a stable division of labor that:
- reduces token usage
- prevents agent drift
- preserves agent judgment where it matters

---

2) Core Principle
- **Control plane owns mechanics.**
- **Agent owns judgment.**
- Agent speaks only at decision points; execution is runner output.

---

3) Control Plane Responsibilities (Owns)

3.1 Deterministic Execution
- step graphs per mode
- runner lifecycle
- non-interactive execution

3.2 Enforcement & Safety
- default constraints per mode
- global + per-mode locked paths
- override permissions
- allow_writes enforcement and scope enforcement (as implemented)

3.3 Definition of Done
- required_proof per mode
- PASS/FAIL/WARN semantics recorded in report.json

3.4 Target Truth
- runtime_contract commands per target (verify/build/ci/rebuild)
- capability detection and environment normalization

3.5 Artifact Production
- run folder creation
- runspec finalization
- runlog writing
- report.json generation
- optional artifacts (diff/snapshot/changelog/plans)

---

4) Agent Responsibilities (Owns)

4.1 Mode Selection
- choose correct MM-xx for the situation

4.2 Scope Selection
- choose scope type and scope boundaries
- prefer components-to-path mapping when available

4.3 Decision-Point Reasoning
- interpret failures (MM-20)
- plan dependency closure (MM-05/MM-08)
- choose recovery strategy (MM-21)
- decide next run in the loop

4.4 Communication
- summarize strictly from report.json
- propose the next action in minimal steps unless more detail is requested

---

5) Shared Interface (Handshake)

5.1 Inputs (Agent → Control Plane)
- pre-run runspec.json
- dispatcher invocation command

5.2 Outputs (Control Plane → Agent)
- report.json (truth)
- runlog.md (evidence)
- artifacts/ (diff/snapshot/changelog/plan docs)

5.3 Operating Loop
1) agent writes runspec
2) agent runs dispatcher
3) control plane produces artifacts
4) agent reads report.json and chooses next run or creates a plan

---

6) Token Control Rules

6.1 Execution is not reasoning
Execution should not be narrated by the agent.
Runners execute; agent reads outcomes.

6.2 Minimize reasoning to decision points
Agent reasoning is allowed only for:
- selecting mode/scope
- interpreting ambiguous failures
- writing required plans (upgrade/migration/recovery)

6.3 Low-token response format (recommended)
For mechanical modes (verify/snapshot/ci):
- produce runspec
- produce dispatcher command
- after run: 3-line summary from report.json

---

7) “What goes where” (Ownership Matrix)

- policies.json: global enforcement + override permissions
- modes.json: required proofs + default constraints + runner mapping
- targets.json: repo commands + expected layout
- runspec.json: run intent (target, scope, notes, allowed overrides)
- report.json: truth (status/results/proof)

---

8) Non-Goals
- This contract does not remove agent judgment.
- It removes repeated procedural narration and boilerplate planning.

Tab 51